

Controlled Measure-valued Martingales and Associated Machine Learning Methods

JOEL BASSIL



Supervised by Chaorui Wang
August 21, 2025
SAMBa Summer Placement

Abstract

This report presents an implementation and comprehensive analysis of the deep 2BSDE (second-order backward stochastic differential equation) method for solving stochastic optimal control problems and their associated Hamilton-Jacobi-Bellman (HJB) equations. We begin with a thorough exposition of Brownian motion, stochastic calculus, the Itô formula, and the Euler-Maruyama discretization scheme, establishing the theoretical foundation for understanding second-order BSDEs and their connection to fully nonlinear parabolic partial differential equations.

We first validate our implementation against a stochastic linear-quadratic control problem with known analytical solutions, demonstrating excellent convergence properties with approximation errors on the order of 10^{-3} . We then conduct dimension scaling analysis up to $d = 100$, revealing exponential growth in computational time and linear scaling in loss with increasing dimension.

This understanding lays the foundation to substitute the Itô process for measure-valued martingale processes. We implement a three-dimensional process supported on $\{-1, 0, 1\}$ with a sophisticated cost function incorporating variance and covariance terms, while enforcing constraints to ensure the process remains within the simplex domain. We provide a detailed mathematical analysis of the discretization scheme's recursion relation, which reveals why loss inevitably becomes large for certain parameter combinations. Through examination of the update equations, we demonstrate that the discrete approximation exhibits exponential growth in the final time T , leading to systematic errors that cannot be eliminated through parameter tuning alone.

We also compare control optimization algorithms, finding that while increasing iterations improves early-stage control approximation, particularly for the first time-step, L-BFGS outperforms ADAM significantly over all time steps. Additional investigations cover discretization parameter optimization and boundary absorption effects.

Finally, we demonstrate applications in mathematical finance by solving the classical Merton portfolio optimization problem, and how to interpret the results of the algorithm.

An interesting take-away from the simulations in the project is that controlling the data process within the 2BSDE seems to pose no issues. In the method construction, the data process takes an auxiliary control, since it is actually irrelevant to the success of the algorithm. If we instead run the simulations but use the optimal control in the update equations at each time step instead of the auxiliary control, the results look almost identical. However, one of our investigations indicated that the further away the auxiliary control is from the optimal control, the longer it took for convergence of loss. Hence, it would seem that using the optimal control instead of the auxiliary control and essentially using modified *C2BSDE* (controlled 2BSDE) method, we are likely to see faster convergence. It appears that this approach has not been seen before, so a rigorous construction of the method and proof that a solution still exists and is unique may be an interesting direction to pursue.

Contents

1	Brownian Motion and an Introduction to Stochastic Calculus	5
1.1	Brownian Motion	5
1.1.1	Construction Via Symmetric Random Walk	5
1.1.2	Construction In Practice	7
1.1.3	Quadratic Variation	7
1.1.4	Filtration and Martingales	7
1.2	Stochastic Differential Equations	8
1.2.1	Itô Integrals For Simple Paths	8
1.2.2	Itô Integrals For General Integrands	9
1.2.3	Itô Formula	9
1.2.4	Itô Processes	10
1.2.5	Multivariable Stochastic Calculus	11
1.3	Euler-Maruyama Scheme	13
2	Deep 2BSDE Method - Brownian Motion Background Dynamics With A Simple ML Technique	15
2.1	Problem Setup	15
2.2	Connection to Second-Order BSDEs	15
2.3	Merged Formulation	16
2.4	Forward Discretization	16
2.5	Neural Network Approximation	16
2.6	Optimization Problem	17
2.7	Itô Process Equivalent Algorithm Formulation	17
3	HJB and 2BSDE Setup	18
3.1	The Problem	18
3.2	Proving 2BSDE (Forward Direction) Using Itô's Formula	19
4	Implementation and Examples: Testing Against Analytical Solutions	20
4.1	Analytical Solutions	20
4.1.1	Optimal Control	20
4.1.2	Value Function	21
4.2	Numerical Results	23
4.3	Investigation	25
4.3.1	Varying Dimension	25
4.3.2	Varying A Constant Control - Effect on Loss	26
5	Measure-valued Martingale Process	28
5.1	Integration With <i>deep 2BSDE</i>	28
5.2	Associated Value Function and HJB Equation	30
5.3	Implementation and Results	30
5.4	Investigation	31
5.4.1	Varying T and N	31
5.4.2	Optimizing Input Parameters	33
5.4.3	Loss Explosion	34
5.4.4	On Early Optimal Control	38
5.4.5	Allowing Bound-Absorption in the Simplex	41

6	Financial Applications of the <i>deep 2BSDE</i> Method	43
6.1	The Merton Problem	43
6.1.1	Implementation with <i>deep 2BSDE</i>	43
A	Appendix - Python Implementation	46
A.1	Brownian Motion SRW Construction and Plotting	46
A.2	Brownian Motion in Practice	47
A.3	Euler Maruyama	47
A.3.1	Example 1	47
A.3.2	Example 2	48
A.4	Example Implementation from Section 4	49
A.5	3-dimension Measure-valued <i>deep 2BSDE</i> Method	61
A.6	Merton Example	75

1 Brownian Motion and an Introduction to Stochastic Calculus

1.1 Brownian Motion

Brownian Motion is a continuous-time stochastic process where the position changes are normally distributed with variance proportional to time, and increments over non-overlapping intervals are independent.

We will first construct it via a scaled symmetric random walk for the purposes of understanding its nature. In practice, we will construct it differently.

1.1.1 Construction Via Symmetric Random Walk

Firstly, we construct a symmetric random walk. Denote a sequence of successive outcomes by X , where $\|X\| = N$ and, for all $j = 1, 2, \dots, N$,

$$X_j = \begin{cases} 1 & \text{with } p = \frac{1}{2} \\ -1 & \text{with } p = \frac{1}{2} \end{cases}$$

Then, define M_k as a symmetric random walk with steps X , as

$$M_j := \sum_{j=1}^k X_j, \quad k = 1, 2, \dots, N \text{ and } M_0 := 0.$$

We define a function to plot the random walk as in [A.1](#). Now, to approximate a Brownian Motion, we speed up time and scale down the size of the random walk. Fix $n > 0$, an integer, and define

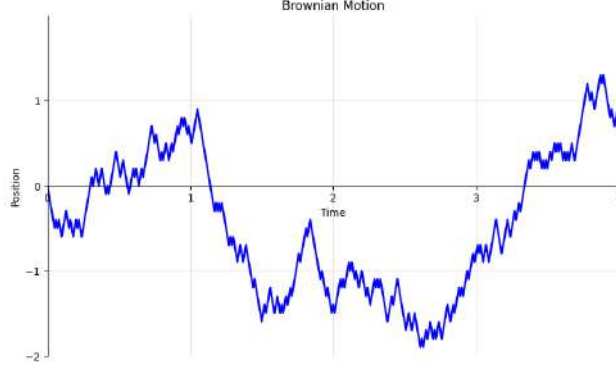
the scaled random walk as

$$W^{(n)}(t) = \frac{1}{\sqrt{n}} M_{nt}$$

provided that nt is an integer. If it is not, define $W^{(n)}(t)$ by linear interpolation between the nearest times s, u , to the left and right of t respectively, such that ns, nu are integers.

By taking the floor and ceiling of nt , we find ns and nu since both n and t are positive so nt is increasing in t .

Below is a simulation of $W^{(100)}(4)$, the Python code for which is in [A.1](#)



Definition 1.1.1 (Brownian Motion). *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. For each $\omega \in \Omega$, suppose there is a continuous function $W(t)$, $t \geq 0$ that satisfies $W(0) = 0$ and that depends on ω . Then $W(t)$, $t \geq 0$ is a Brownian Motion if for all $0 = t_0 < t_1 < \dots < t_m$ the increments*

$$W(t_1) - W(t_0), W(t_2) - W(t_1), \dots, W(t_m) - W(t_{m-1}) \quad (1)$$

are independent and each of these increments is normally distributed with

$$\mathbb{E}[W(t_{i+1}) - W(t_i)] = 0 \quad (2)$$

$$\text{Var}[W(t_{i+1}) - W(t_i)] = t_{i+1} - t_i \quad (3)$$

Theorem 1.1.2 (Central Limit). *Fix $t > 0$. As $n \rightarrow \infty$, the distribution of the scaled random walk $W^n(t)$ evaluated at time t converges to the normal distribution with mean zero and variance t .*

For the proof, see Theorem 3.2.1 in [3].

Remark 1.1.3. The construction of Brownian Motion using a scaled random walk fits the criteria of the Brownian Motion definition. Consider $W^{(n)}(t)$.

If $0 = t_0 < t_1 < \dots < t_m$ are such that each nt_j is an integer, then

$$(W^{(n)}(t_1) - W^{(n)}(t_0)), (W^{(n)}(t_2) - W^{(n)}(t_1)), \dots, (W^{(n)}(t_m) - W^{(n)}(t_{m-1}))$$

are independent since these random variables depend on different 'coin tosses'.

Also, $W^{(n)}(t) - W^{(n)}(s)$ is the sum of $n(t - s)$ independent random variables, each with expected value of zero and variance $1/n$. Hence

$$\mathbb{E}[W^{(n)}(t) - W^{(n)}(s)] = 0 \cdot n(t - s) = 0$$

$$\text{Var}[W^{(n)}(t) - W^{(n)}(s)] = \frac{1}{n} \cdot n(t - s) = t - s$$

so (2) and (3) are satisfied. (1) is satisfied using Theorem 1.1.2 since the sum of independent, normally distributed random variables is also normally distributed.

1.1.2 Construction In Practice

In practice, we are not concerned about the entire path of a Brownian Motion, but rather the overall movement between two times. Since, by definition, the Brownian increments are normally distributed, we can construct them using a normal distribution. Over a time period Δt , $W(t) \sim \mathcal{N}(0, \Delta t)$ which satisfies that $\mathbb{E}(W(t)) = 0$ and $\text{Var}(W(t)) = \Delta t$ as per the definition.

The code in [A.2](#) simulates a vector $\mathbf{v}$ such that $\|\mathbf{v}\| = M \in \mathbb{N}$ and every $v_i \in \mathbf{v}$ is a Brownian increment over $[0, T]$, that is, $v_i \sim \mathcal{N}(0, T)$ for all $i = 1, 2, \dots, M$.

1.1.3 Quadratic Variation

Quadratic variation measures the "roughness" or variability of a stochastic process along its sample paths. This will become useful in the stochastic calculus section below.

Definition 1.1.4 (Quadratic Variation). *Let $f(t)$ be a function defined for $0 \leq t \leq T$. The quadratic variation of f up to time T is*

$$[f, f](T) := \lim_{\|\Pi\| \rightarrow 0} \sum_{j=0}^{n-1} [f(t_{j+1}) - f(t_j)]^2, \quad (4)$$

where $\Pi = \{t_0, t_1, \dots, t_n\}$ and $0 = t_0 < t_1 < \dots < t_n = T$.

Theorem 1.1.5. *Let W be a Brownian Motion. Then $[W, W](T) = T$ for all $T \geq 0$ almost surely.*

For the proof, see Theorem 3.4.3 in [\[3\]](#).

1.1.4 Filtration and Martingales

In addition to the Brownian Motion itself, we need the following notation for the amount of information available at each time, called a filtration:

Definition 1.1.6. *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space on which is defined a Brownian Motion $W(t)$, $t \geq 0$. A filtration for the Brownian Motion is a collection of σ -algebras $\mathcal{F}(t)$, $t \geq 0$, satisfying:*

- (i) (**Information accumulates**) *For $0 \leq s < t$, every set in $\mathcal{F}(s)$ is also in $\mathcal{F}(t)$. In other words, there is at least as much information available at the later time $\mathcal{F}(t)$ as there is at the earlier time $\mathcal{F}(s)$.*
- (ii) (**Adaptivity**) *For each $t \geq 0$, the Brownian Motion $W(t)$ at time t is $\mathcal{F}(t)$ -measurable. In other words, the information available at time t is sufficient to evaluate the Brownian Motion $W(t)$ at that time.*
- (iii) (**Independence of future increments**) *For $0 \leq t < u$, the increment $W(u) - W(t)$ is independent of $\mathcal{F}(t)$. In other words, any increment of the Brownian Motion after time t is independent of the information available at time t .*

Let $\Delta(t)$, $t \geq 0$, be a stochastic process. We say that $\Delta(t)$ is adapted to the filtration $\mathcal{F}(t)$ if for each $t \geq 0$ the random variable $\Delta(t)$ is $\mathcal{F}(t)$ -measurable.

Definition 1.1.7 (Martingale). *A stochastic process $\{X_t\}_{t \geq 0}$ adapted to a filtration $\{\mathcal{F}_t\}_{t \geq 0}$ is a martingale if $\mathbb{E}[|X_t|] < \infty$ for all $t \geq 0$ and*

$$\mathbb{E}[X_t | \mathcal{F}_s] = X_s$$

for all $0 \leq s \leq t$.

Theorem 1.1.8. *Brownian Motion is a martingale.*

For the proof, see Theorem 3.3.4 in [3].

Theorem 1.1.9 (Markov Property). *Let $W(t)$, $t \geq 0$, be a Brownian Motion and let $\mathcal{F}(t)$, $t \geq 0$, be a filtration for this Brownian Motion (see Definition 3.3.3). Then $W(t)$, $t \geq 0$, is a Markov process.*

For the proof, see Theorem 3.5.1 in [3].

1.2 Stochastic Differential Equations

1.2.1 Itô Integrals For Simple Paths

Itô Integrals are used to model the value of a portfolio that results from trading assets in continuous time. The calculus used to manipulate these is based on the Itô formula.

We start by fixing a positive number T and seeking to make sense of

$$\int_0^T \Delta(t) dW(t) \quad (5)$$

where $W(t)$, $t \geq 0$ is a Brownian Motion with a filtration $\mathcal{F}(t)$, $t \geq 0$ and $\Delta(t)$ is an adapted stochastic process. We can think of $\Delta(t)$ as eventually being the position we take in an asset at time t . This will depend on the price path of the asset up to time t .

The issue with trying to assign meaning to the above integral is that Brownian Motion paths cannot be differentiated with respect to time. Instead, to make sense of the integral, we define the Itô integral for simple paths and then extend it.

Let $\Pi = \{t_0, t_1, \dots, t_n\}$ be a partition of $[0, T]$, that is, $0 = t_0 \leq t_1 \leq \dots \leq t_n = T$. Assume that $\Delta(t)$ is constant in t on each subinterval $[t_j, t_{j+1})$. $\Delta(t)$ is a simple process.

Think of $W(t)$ as the PPS (price per share) of an asset at time t ; t_j as a trading date; and $\Delta(t_j)$ as the position taken in the asset on trading date t_j . Then the gain at time t , if $t_k \leq t \leq t_{k+1}$, is

$$I(t) = \sum_{j=0}^{k-1} \Delta(t_j)[W(t_{j+1}) - W(t_j)] + \Delta(t_k)[W(t) - W(t_k)]. \quad (6)$$

which is the sum of the gains of trading between each trading date. (6) is the Itô integral of the simple process $\Delta(t)$, which we write as

$$I(t) = \int_0^t \Delta(u) dW(u), \quad (7)$$

as in (5).

Theorem 1.2.1. *The Itô integral in (7) is a martingale.*

For the proof, see Theorem 4.2.1 in [3].

Theorem 1.2.2. *The quadratic variation accumulated up to time t by the Itô integral (7) is*

$$[I, I](t) = \int_0^t \Delta^2(u) du. \quad (8)$$

For the proof, see Theorem 4.2.3 in [3].

We can see from this result that quadratic variation of an Itô integral can depend on the path. That is, if we choose large positions $\Delta(u)$, the quadratic variation will be large. The negation is also true. Hence, quadratic variation can be regarded as a measure of risk, and is dependent on the size of positions taken.

1.2.2 Itô Integrals For General Integrand

We no longer assume that $\Delta(t)$ is a simple process, but we can choose a sequence $\Delta_n(t)$ of simple processes which converge to a continuously varying $\Delta(t)$ as $n \rightarrow \infty$. Hence we have the following definition:

Definition 1.2.3.

$$I(t) := \int_0^t \Delta(u) dW(u) = \lim_{n \rightarrow \infty} \int_0^t \Delta_n(u) dW(u), \quad 0 \leq t \leq T. \quad (9)$$

This integral inherits the properties of the integral for simple processes.

Theorem 1.2.4. *Let T be a positive constant and let $\Delta(t)$, $0 \leq t \leq T$, be an adapted stochastic process that satisfies (4.3.1). Then $I(t) = \int_0^t \Delta(u) dW(u)$ defined by (6) has the following properties.*

- (i) **(Continuity)** *As a function of the upper limit of integration t , the paths of $I(t)$ are continuous.*
- (ii) **(Adaptivity)** *For each t , $I(t)$ is $\mathcal{F}(t)$ -measurable.*
- (iii) **(Linearity)** *If $I(t) = \int_0^t \Delta(u) dW(u)$ and $J(t) = \int_0^t \Gamma(u) dW(u)$, then*

$$I(t) \pm J(t) = \int_0^t (\Delta(u) \pm \Gamma(u)) dW(u);$$

furthermore, for every constant c ,

$$cI(t) = \int_0^t c\Delta(u) dW(u).$$

- (iv) **(Martingale)** *$I(t)$ is a martingale.*
- (v) **(Itô isometry)** $\mathbb{E}I^2(t) = \mathbb{E} \int_0^t \Delta^2(u) du.$
- (vi) **(Quadratic variation)** $[I, I](t) = \int_0^t \Delta^2(u) du.$

1.2.3 Itô Formula

We want to be able to integrate expressions of the form $f(W(t))$ where f is differentiable and $W(t)$ is a Brownian Motion. If $W(t)$ were also differentiable, we could use the chain rule to give

$$df(W(t)) := \frac{d}{dt}f(W(t)) = f'(W(t))W'(t) := f'(W(t))dW(t)$$

Since W has nonzero quadratic variation, the correct expression has an extra term. This is because the quadratic variation in Brownian Motion is exactly T , meaning it is not negligible compared to the first term as $t \rightarrow 0$. The extra term is:

$$df(W(t)) = f'(W(t))dW(t) + \frac{1}{2}f''(W(t))dt.$$

This is the Itô formula in differential form. In integral form, it is:

$$f(W(t)) - f(W(0)) = \int_0^t f'(W(u))dW(u) + \frac{1}{2} \int_0^t f''(W(u))du$$

This is mathematically meaningful since the first term is an Itô integral, defined earlier, and the second is an ordinary Lebesgue integral with respect to t .

Theorem 1.2.5 (Itô formula for Brownian Motion). *Let $f(t, x)$ be a function for which the partial derivatives $f_t(t, x)$, $f_x(t, x)$, and $f_{xx}(t, x)$ are defined and continuous, and let $W(t)$ be a Brownian Motion. Then, for every $T \geq 0$,*

$$\begin{aligned} f(T, W(T)) &= f(0, W(0)) + \int_0^T f_t(t, W(t)) dt \\ &\quad + \int_0^T f_x(t, W(t)) dW(t) + \frac{1}{2} \int_0^T f_{xx}(t, W(t)) dt. \end{aligned} \tag{10}$$

For the proof, see Theorem 4.4.1 in [3].

In differential notation, this is

$$df(t, W(t)) = f_t(t, W(t)) dt + f_x(t, W(t)) dW(t) + \frac{1}{2} f_{xx}(t, W(t)) dt. \tag{11}$$

1.2.4 Itô Processes

We can extend the Itô formula to stochastic processes (without jumps) more general than Brownian Motion. These are called Itô processes.

Definition 1.2.6 (Itô Process). *Let $W(t)$, $t \geq 0$, be a Brownian Motion, and let $\mathcal{F}(t)$, $t \geq 0$, be an associated filtration. An Itô process is a stochastic process of the form*

$$X(t) = X(0) + \int_0^t \Delta(u) dW(u) + \int_0^t \Theta(u) du, \tag{12}$$

where $X(0)$ is nonrandom and $\Delta(u)$ and $\Theta(u)$ are adapted stochastic processes.

In order to understand the volatility associated with Itô processes, we must determine the rate at which they accumulate quadratic variation.

Lemma 1.2.7. *The quadratic variation of the Itô process (12) is*

$$[X, X](t) = \int_0^t \Delta^2(u) du. \quad (13)$$

For the proof, see Lemma 4.4.4 in [3].

1.2.5 Multivariable Stochastic Calculus

We will now explore multi-dimensional Brownian Motions.

Definition 1.2.8. *A d -dimensional Brownian Motion is a process*

$$W(t) = (W_1(t), \dots, W_d(t))$$

with the following properties.

- (i) *Each $W_i(t)$ is a one-dimensional Brownian Motion.*
- (ii) *If $i \neq j$, then the processes $W_i(t)$ and $W_j(t)$ are independent.*

Associated with a d -dimensional Brownian Motion, we have a filtration $\mathcal{F}(t)$, $t \geq 0$, such that the following holds.

- (iii) **(Information accumulates)** *For $0 \leq s < t$, every set in $\mathcal{F}(s)$ is also in $\mathcal{F}(t)$.*
- (iv) **(Adaptivity)** *For each $t \geq 0$, the random vector $W(t)$ is $\mathcal{F}(t)$ -measurable.*
- (v) **(Independence of future increments)** *For $0 \leq t < u$, the vector of increments $W(u) - W(t)$ is independent of $\mathcal{F}(t)$.*

Remark 1.2.9. Each W_i is a one-dimensional Brownian Motion so the quadratic variation is $[W_i, W_i](t) = t$. If $i \neq j$ then independence of W_i and W_j implies $[W_i, W_j](t) = 0$, $\forall i \neq j$. For the proof, see Section 4.6.1 in [3].

We can now derive and define the Itô formula for multiple processes. We will write it for two processes driven by a 2-d Brownian Motion. This then generalizes to any number of processes and dimensions.

Let $X(t)$ and $Y(t)$ be Itô processes, which means they are processes of the form

$$X(t) = X(0) + \int_0^t \Theta_1(u) du + \int_0^t \sigma_{11}(u) dW_1(u) + \int_0^t \sigma_{12}(u) dW_2(u),$$

$$Y(t) = Y(0) + \int_0^t \Theta_2(u) du + \int_0^t \sigma_{21}(u) dW_1(u) + \int_0^t \sigma_{22}(u) dW_2(u).$$

The integrands $\Theta_i(u)$ and $\sigma_{ij}(u)$ are assumed to be adapted processes. In differential notation, we write

$$dX(t) = \Theta_1(t) dt + \sigma_{11}(t) dW_1(t) + \sigma_{12}(t) dW_2(t), \quad (14)$$

$$dY(t) = \Theta_2(t) dt + \sigma_{21}(t) dW_1(t) + \sigma_{22}(t) dW_2(t). \quad (15)$$

The Itô integral $\int_0^t \sigma_{11}(u) dW_1(u)$ accumulates quadratic variation at rate $\sigma_{11}^2(t)$ per unit time, and the Itô integral $\int_0^t \sigma_{12}(u) dW_2(u)$ accumulates quadratic variation at rate $\sigma_{12}^2(t)$ per unit time.

Because both of these integrals appear in $X(t)$, the process $X(t)$ accumulates quadratic variation at rate $\sigma_{11}^2(t) + \sigma_{12}^2(t)$ per unit time:

$$[X, X](t) = \int_0^t (\sigma_{11}^2(u) + \sigma_{12}^2(u)) du.$$

We may write this equation in differential form as

$$dX(t) dX(t) = (\sigma_{11}^2(t) + \sigma_{12}^2(t)) dt. \quad (16)$$

One can informally derive this by squaring the first equation and using the multiplication rules

$$dt dt = 0, \quad dt dW_i(t) = 0, \quad dW_i(t) dW_i(t) = dt, \quad dW_i(t) dW_j(t) = 0 \text{ for } i \neq j.$$

In a similar way, we may derive the differential formulas

$$dY(t) dY(t) = (\sigma_{21}^2(t) + \sigma_{22}^2(t)) dt, \quad (17)$$

$$dX(t) dY(t) = (\sigma_{11}(t)\sigma_{21}(t) + \sigma_{12}(t)\sigma_{22}(t)) dt. \quad (18)$$

Equation (18) says that, for every $T \geq 0$,

$$[X, Y](T) = \int_0^T (\sigma_{11}(t)\sigma_{21}(t) + \sigma_{12}(t)\sigma_{22}(t)) dt. \quad (19)$$

Definition 1.2.10 (Cross Variation). *The term $[X, Y](T)$ on the left-hand side is defined as follows. Let $\Pi = \{t_0, t_1, \dots, t_n\}$ be a partition of $[0, T]$ (i.e., $0 = t_0 < t_1 < \dots < t_n = T$) and set up the sampled cross variation*

$$\sum_{k=0}^{n-1} [X(t_{k+1}) - X(t_k)][Y(t_{k+1}) - Y(t_k)]. \quad (20)$$

Now let the number of partition points n go to infinity as the length of the longest subinterval $\|\Pi\| = \max_{0 \leq k \leq n-1} (t_{k+1} - t_k)$ goes to zero. The limit of the sum is $[X, Y](T)$, which is given by the RHS of (19).

The following theorem generalizes the Itô formula of Theorem 1.2.5 to the case of any pair of Itô processes in two dimensions:

Theorem 1.2.11 (Two-dimensional Itô Formula). *Let $f(t, x, y)$ be a function whose partial derivatives $f_t, f_x, f_y, f_{xx}, f_{xy}, f_{yx},$ and f_{yy} are defined and are continuous. Let $X(t)$ and $Y(t)$ be Itô processes as discussed above. The two-dimensional Itô formula in differential form is*

$$\begin{aligned} df(t, X(t), Y(t)) &= f_t(t, X(t), Y(t)) dt + f_x(t, X(t), Y(t)) dX(t) + f_y(t, X(t), Y(t)) dY(t) \\ &\quad + \frac{1}{2} f_{xx}(t, X(t), Y(t)) dX(t) dX(t) + f_{xy}(t, X(t), Y(t)) dX(t) dY(t) \\ &\quad + \frac{1}{2} f_{yy}(t, X(t), Y(t)) dY(t) dY(t). \end{aligned} \quad (21)$$

We rewrite it, leaving out t wherever possible, to obtain the same formula in the more compact notation

$$df(t, X, Y) = f_t dt + f_x dX + f_y dY + \frac{1}{2} f_{xx} dX dX + f_{xy} dX dY + \frac{1}{2} f_{yy} dY dY. \quad (22)$$

In integral form, this is

$$\begin{aligned} & f(t, X(t), Y(t)) - f(0, X(0), Y(0)) \\ &= \int_0^t [\sigma_{11}(u) f_x(u, X(u), Y(u)) + \sigma_{21}(u) f_y(u, X(u), Y(u))] dW_1(u) \\ &+ \int_0^t [\sigma_{12}(u) f_x(u, X(u), Y(u)) + \sigma_{22}(u) f_y(u, X(u), Y(u))] dW_2(u) \\ &+ \int_0^t [f_t(u, X(u), Y(u)) \\ &+ \Theta_1(u) f_x(u, X(u), Y(u)) + \Theta_2(u) f_y(u, X(u), Y(u)) \\ &+ \frac{1}{2} (\sigma_{11}^2(u) + \sigma_{12}^2(u)) f_{xx}(u, X(u), Y(u)) \\ &+ (\sigma_{11}(u) \sigma_{21}(u) + \sigma_{12}(u) \sigma_{22}(u)) f_{xy}(u, X(u), Y(u)) \\ &+ \frac{1}{2} (\sigma_{21}^2(u) + \sigma_{22}^2(u)) f_{yy}(u, X(u), Y(u))] du. \end{aligned} \quad (23)$$

which can be found by substituting (16)–(18) into (22).

For a single process X_t which is a d -dimensional vector, the Itô formula is

$$\begin{aligned} df(t, X) &= f_t dt + \sum_{i=1}^d f_{x_i} dX_t^i + \frac{1}{2} \sum_{i,j=1}^d f_{x_i x_j} dX_t^i dX_t^j \\ &= f_t dt + \nabla f \cdot dX_t + \frac{1}{2} \text{Tr}(H f \sigma \sigma^\top) dt \end{aligned} \quad (24)$$

with the $\sigma \sigma^\top$ and dt at the end appearing due to the quadratic cross variation and swapping of indices.

1.3 Euler-Maruyama Scheme

The Euler-Maruyama method is a method for the approximate numerical solution of a stochastic differential equation.

Definition 1.3.1 (Euler-Maruyama Approximation). *Consider the stochastic differential equation*

$$dX_t = \Theta(X_t, t) dt + \sigma(X_t, t) dW_t,$$

*with initial condition $X_0 = x_0$, where W_t is a Brownian Motion, and suppose that we wish to solve this SDE on some interval of time $[0, T]$. Then the **Euler-Maruyama approximation** to the true solution X is the Markov chain Y defined as follows:*

- Partition the interval $[0, T]$ into N equal subintervals of width $\Delta t > 0$:

$$0 = t_0 < t_1 < \cdots < t_N = T \quad \text{and} \quad \Delta t = T/N;$$

- Set $Y_0 = x_0$
- Recursively define Y_n for $0 \leq n \leq N - 1$ by

$$Y_{n+1} = Y_n + \Theta(Y_n, t_n)\Delta t + \sigma(Y_n, t_n)\Delta W_n,$$

where

$$\Delta W_n = W_{t_{n+1}} - W_{t_n}.$$

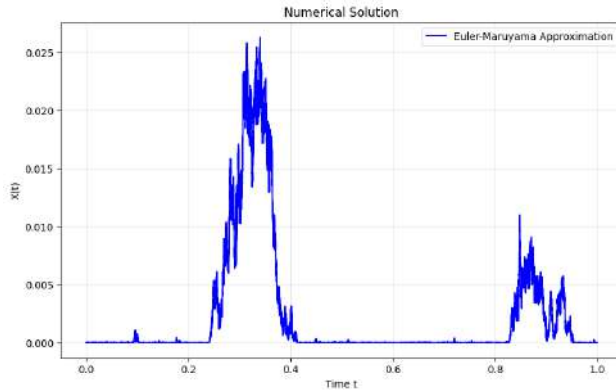
Example 1.3.2. Consider the stochastic differential equation:

$$dX_t = \frac{1}{2}\sqrt{X_t}dW_t, \quad X_0 = 0 \quad (25)$$

where W_t is a standard Brownian Motion.

Immediately, $X_t = 0$ is a solution. Intuitively, this is the only solution, since there is no drift term. We can show this visually using the Euler-Maruyama method, and add perturbations to simulate the return to 0. Without perturbations, it would never leave 0.

We define a function simulating this approximation and plot it using the code in [A.3.1](#).



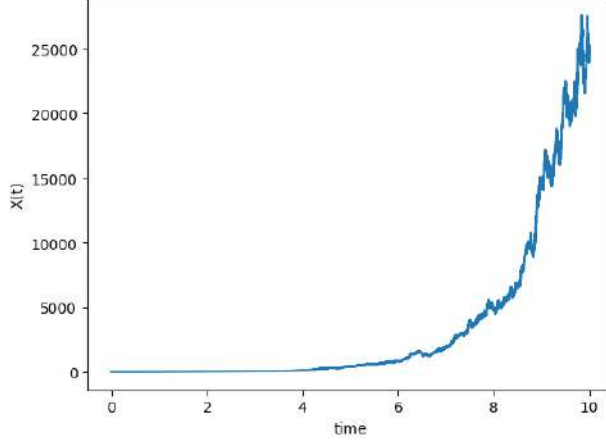
Example 1.3.3. Consider now the process

$$dX_t = (u_t + X_t)dt + \left(\frac{1}{2}u_t - \frac{1}{3}X_t\right)dW_t, \quad X_0 = 0 \quad (26)$$

Let u be a constant control process, i.e. $u_t \equiv a \in \mathbb{R}$ for all $t \in [0, T]$. We can apply the Euler-Maruyama scheme by writing a function to simulate M paths of the process X and output them as a single numpy array.

We can write reusable code for this simulation, by creating functions for the drift term $(b(u, x))$ and the diffusion term $(\sigma(u, x))$. We can also use the function defined earlier which simulates M samples of Brownian increments. The code can be found in [A.3.2](#).

Below is a plot of one path of this process:



2 Deep 2BSDE Method - Brownian Motion Background Dynamics With A Simple ML Technique

The deep 2BSDE method addresses fully nonlinear second-order parabolic PDEs of the following form. For proofs and reasoning, see [1], Section 2, where the method is derived. A more general case where the background process can be any Itô process and the scheme can employ more sophisticated ML methods can be found in Section 3 of [1].

2.1 Problem Setup

Let $d \in \mathbb{N}$, $T \in (0, \infty)$, and consider functions $f : [0, T] \times \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \rightarrow \mathbb{R}$ and $g : \mathbb{R}^d \rightarrow \mathbb{R}$. We seek a function $u : [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}$ satisfying:

$$u(T, x) = g(x), \quad x \in \mathbb{R}^d \quad (27)$$

$$\frac{\partial u}{\partial t}(t, x) = f(t, x, u(t, x), (\nabla_x u)(t, x), (\text{Hess}_x u)(t, x)) \quad (28)$$

for all $(t, x) \in [0, T) \times \mathbb{R}^d$.

2.2 Connection to Second-Order BSDEs

The key insight is the probabilistic representation via second-order backward stochastic differential equations (2BSDEs).

Under suitable smoothness and regularity conditions, let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space with standard Brownian motion $W : [0, T] \times \Omega \rightarrow \mathbb{R}^d$ and natural filtration $\mathbb{F} = (\mathcal{F}_t)_{t \in [0, T]}$. For $\xi \in \mathbb{R}^d$, define $\mathbb{F}$ -adapted processes $Y, Z, \Gamma, A : [0, T] \times \Omega \rightarrow \mathbb{R}, \mathbb{R}^d, \mathbb{R}^{d \times d}, \mathbb{R}^d$ by:

$$Y_t = u(t, \xi + W_t) \quad (29)$$

$$Z_t = (\nabla_x u)(t, \xi + W_t) \quad (30)$$

$$\Gamma_t = (\text{Hess}_x u)(t, \xi + W_t) \quad (31)$$

$$A_t = \left(\frac{\partial}{\partial t} \nabla_x u \right) (t, \xi + W_t) + \frac{1}{2} (\nabla_{xx} u)(t, \xi + W_t) \quad (32)$$

Then these processes relate the PDE (27), (28) to the 2BSDE system as follows:

$$Y_t = g(\xi + W_T) - \int_t^T \left[f(s, \xi + W_s, Y_s, Z_s, \Gamma_s) + \frac{1}{2} \text{Trace}(\Gamma_s) \right] ds - \int_t^T \langle Z_s, dW_s \rangle \quad (33)$$

$$Z_t = Z_0 + \int_0^t A_s ds + \int_0^t \Gamma_s dW_s \quad (34)$$

2.3 Merged Formulation

To develop a numerical scheme, we merge the PDE and 2BSDE formulations.

Observe that (33) and (34) yield that for any $\tau_1, \tau_2 \in [0, T]$ with $\tau_1 \leq \tau_2$,

$$\begin{aligned} Y_{\tau_2} = Y_{\tau_1} &+ \int_{\tau_1}^{\tau_2} \langle Z_s, dW_s \rangle + \int_{\tau_1}^{\tau_2} \left[f(s, \xi + W_s, Y_s, Z_s, (\text{Hess}_x u)(s, \xi + W_s)) \right. \\ &\left. + \frac{1}{2} \text{Trace}((\text{Hess}_x u)(s, \xi + W_s)) \right] ds \end{aligned} \quad (35)$$

$$\begin{aligned} Z_{\tau_2} = Z_{\tau_1} &+ \int_{\tau_1}^{\tau_2} \left[\left(\frac{\partial}{\partial t} \nabla_x u \right) (s, \xi + W_s) + \frac{1}{2} (\nabla_{xx} u)(s, \xi + W_s) \right] ds \\ &+ \int_{\tau_1}^{\tau_2} (\text{Hess}_x u)(s, \xi + W_s) dW_s \end{aligned} \quad (36)$$

2.4 Forward Discretization

Let $0 = t_0 < t_1 < \dots < t_N = T$ be a time discretization. The forward Euler-Maruyama scheme suggests that

For $n \in \{0, 1, \dots, N-1\}$:

$$\begin{aligned} Y_{t_{n+1}} &\approx Y_{t_n} + f(t_n, \xi + W_{t_n}, Y_{t_n}, Z_{t_n}, (\text{Hess}_x u)(t_n, \xi + W_{t_n}))(t_{n+1} - t_n) \\ &+ \frac{1}{2} \text{Trace}((\text{Hess}_x u)(t_n, \xi + W_{t_n}))(t_{n+1} - t_n) + \langle Z_{t_n}, W_{t_{n+1}} - W_{t_n} \rangle \end{aligned} \quad (37)$$

$$\begin{aligned} Z_{t_{n+1}} &\approx Z_{t_n} + \left[\left(\frac{\partial}{\partial t} \nabla_x u \right) (t_n, \xi + W_{t_n}) + \frac{1}{2} (\nabla_{xx} u)(t_n, \xi + W_{t_n}) \right] (t_{n+1} - t_n) \\ &+ (\text{Hess}_x u)(t_n, \xi + W_{t_n})(W_{t_{n+1}} - W_{t_n}) \end{aligned} \quad (38)$$

2.5 Neural Network Approximation

The key insight is to approximate the unknown functions using deep neural networks.

For parameter vector $\theta \in \mathbb{R}^\nu$, $\nu \in \mathbb{N} \cap [d+1, \infty]$ and $n \in \{0, 1, \dots, N-1\}$, let:

- $\mathcal{G}_n^\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$ approximate $(\text{Hess}_x u)(t_n, \cdot)$
- $\mathcal{A}_n^\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ approximate $\left(\frac{\partial}{\partial t} \nabla_x u \right) (t_n, \cdot) + \frac{1}{2} (\nabla_{xx} u)(t_n, \cdot)$

Define stochastic processes $\mathcal{Y}^\theta, \mathcal{Z}^\theta$ with initial conditions $\mathcal{Y}_0^\theta = \theta_1$ and $\mathcal{Z}_0^\theta = (\theta_2, \dots, \theta_{d+1})$, evolving by:

$$\begin{aligned} \mathcal{Y}_{n+1}^\theta &= \mathcal{Y}_n^\theta + \langle \mathcal{Z}_n^\theta, W_{t_{n+1}} - W_{t_n} \rangle \\ &+ \left[f(t_n, \xi + W_{t_n}, \mathcal{Y}_n^\theta, \mathcal{Z}_n^\theta, \mathcal{G}_n^\theta(\xi + W_{t_n})) + \frac{1}{2} \text{Trace}(\mathcal{G}_n^\theta(\xi + W_{t_n})) \right] (t_{n+1} - t_n) \end{aligned} \quad (39)$$

$$\mathcal{Z}_{n+1}^\theta = \mathcal{Z}_n^\theta + \mathcal{A}_n^\theta(\xi + W_{t_n})(t_{n+1} - t_n) + \mathcal{G}_n^\theta(\xi + W_{t_n})(W_{t_{n+1}} - W_{t_n}) \quad (40)$$

For all suitable $\theta \in \mathbb{R}^\nu$ and all $n \in \{0, 1, \dots, N\}$ we think of $\mathcal{Y}_n^\theta : \Omega \rightarrow \mathbb{R}$ as an appropriate approximation

$$\mathcal{Y}_n^\theta \approx Y_{t_n} \quad (41)$$

of $Y_{t_n} : \Omega \rightarrow \mathbb{R}$. Moreover, for all suitable $\theta \in \mathbb{R}^\nu$ and all $n \in \{0, 1, \dots, N\}$ we think of $\mathcal{Z}_n^\theta : \Omega \rightarrow \mathbb{R}^d$ as an appropriate approximation

$$\mathcal{Z}_n^\theta \approx Z_{t_n} \quad (42)$$

In particular, we think of $\theta_1 \in \mathbb{R}$ as an appropriate approximation

$$\theta_1 \approx u(0, \xi) \quad (43)$$

of $u(0, \xi) \in \mathbb{R}$, and we think of $(\theta_2, \theta_3, \dots, \theta_{d+1}) \in \mathbb{R}^d$ as an appropriate approximation

$$(\theta_2, \theta_3, \dots, \theta_{d+1}) \approx (\nabla_x u)(0, \xi) \quad (44)$$

of $(\nabla_x u)(0, \xi) \in \mathbb{R}^d$.

The following table summarizes the approximations made:

Object which we intend to approximate	Approximation
$u(t_n, \xi + W_{t_n}) = Y_{t_n} : \Omega \rightarrow \mathbb{R}$	$\mathcal{Y}_n^\theta : \Omega \rightarrow \mathbb{R}$
$(\nabla_x u)(t_n, \xi + W_{t_n}) = Z_{t_n} : \Omega \rightarrow \mathbb{R}^d$	$\mathcal{Z}_n^\theta : \Omega \rightarrow \mathbb{R}^d$
$(\text{Hess}_x u)(t_n, x) \in \mathbb{R}^{d \times d}, x \in \mathbb{R}^d,$	$G_n^\theta(x) \in \mathbb{R}^{d \times d}, x \in \mathbb{R}^d,$
$(\text{Hess}_x u)(t_n, \xi + W_{t_n}) = \Gamma_{t_n} : \Omega \rightarrow \mathbb{R}^{d \times d}$	$G_n^\theta(\xi + W_{t_n}) : \Omega \rightarrow \mathbb{R}^{d \times d}$
$(\frac{\partial}{\partial t} \nabla_x u)(t_n, x) + \frac{1}{2}(\nabla_x \Delta_x u)(t_n, x) \in \mathbb{R}^d, x \in \mathbb{R}^d,$	$A_n^\theta(x) \in \mathbb{R}^d, x \in \mathbb{R}^d,$
$(\frac{\partial}{\partial t} \nabla_x u)(t_n, \xi + W_{t_n}) + \frac{1}{2}(\nabla_x \Delta_x u)(t_n, \xi + W_{t_n}) = A_{t_n} : \Omega \rightarrow \mathbb{R}^d$	$A_n^\theta(\xi + W_{t_n}) : \Omega \rightarrow \mathbb{R}^d$
$u(0, \xi) \in \mathbb{R}$	$\theta_1 \in \mathbb{R}$
$(\nabla_x u)(0, \xi) \in \mathbb{R}^d$	$(\theta_2, \theta_3, \dots, \theta_{d+1}) \in \mathbb{R}^d$

2.6 Optimization Problem

The neural network parameters are optimized via a stochastic gradient descent-type algorithm. Define the loss function as:

$$L(\theta) := \mathbb{E} \left[|\mathcal{Y}_N^\theta - g(\xi + W_T)|^2 \right]$$

The optimal parameters θ^* solve:

$$\theta^* \in \arg \min_{\theta \in \mathbb{R}^\nu} L(\theta)$$

The approximate solution is then $u(0, \xi) \approx \theta_0^*$.

2.7 Itô Process Equivalent Algorithm Formulation

The formulation thus far has been with the underlying stochastic process as a simple Brownian Motion. Where the underlying process is a more general Itô Process, the update equations change slightly but the neural network structure remains largely the same. The new update equations, equivalent to (39) and (40) in the simple case, are

$$\begin{aligned}\mathcal{Y}_{n+1}^\theta &= \mathcal{Y}_n^\theta + \langle \mathcal{Z}_n^\theta, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle \\ &\quad + \left[f(t_n, \mathcal{X}_n, \mathcal{Y}_n^\theta, \mathcal{Z}_n^\theta, \mathcal{G}_n^\theta(\mathcal{X}_n)) + \frac{1}{2} \text{Trace}(\sigma(\mathcal{X}_n) \sigma^\top(\mathcal{X}_n) \mathcal{G}_n^\theta(\mathcal{X}_n)) \right] (t_{n+1} - t_n)\end{aligned}\quad (45)$$

$$\mathcal{Z}_{n+1}^\theta = \mathcal{Z}_n^\theta + \mathcal{A}_n^\theta(\mathcal{X}_n)(t_{n+1} - t_n) + \mathcal{G}_n^\theta(\mathcal{X}_n)(\mathcal{X}_{n+1} - \mathcal{X}_n) \quad (46)$$

where

$$\mathcal{X}_{n+1} = \mathcal{X}_n + b(u^{\text{aux}}, \mathcal{X}_n)(t_{n+1} - t_n) + \sigma(u^{\text{aux}}, \mathcal{X}_n)(W_{t_{n+1}} - W_{t_n}) \quad (47)$$

These are derived (up to forward discretization) in section 3.2.

3 HJB and 2BSDE Setup

We can now define our problem and proceed to use the deep 2BSDE method to solve it.

We will apply the more general Itô process version, with the backwards optimization using the ADAM optimizer rather than a regular stochastic gradient descent algorithm.

3.1 The Problem

Consider the process

$$dX_t = b(u_t, X_t)dt + \sigma(u_t, X_t)dW_t, \quad X_0 = x_0. \quad (48)$$

The process X_t is said to be controlled by the control process u_t . Let the objective be

$$J(t, u; x_0) = \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(u_s, X_s) ds + e^{-\beta(T-t)} g(X_T) \right]. \quad (49)$$

The function f is usually called the running cost, and the function g is usually called the terminal cost/final cost.

Define the value function as

$$v(t, x) := \inf_u J(t, u; x). \quad (50)$$

Then by the dynamic programming principle, $v(t, x)$ satisfies the so-called Hamilton-Jacobi-Bellman equation,

$$\partial_t v - \beta v + \inf_u \left\{ k(u, x) + b(u, x) \cdot \partial_x v + \frac{1}{2} \text{Tr}(\sigma(u, x) \sigma^\top(u, x) \partial_{xx}^2 v) \right\} = 0, \quad v(T, x) = g(x). \quad (51)$$

That is:

$$-v_t(t, x) + f(t, x, v(t, x), Dv(t, x), D^2v(t, x)) = 0 \quad (52)$$

where $f(t, x, v(t, x), Dv(t, x), D^2v(t, x))$ is defined as:

$$f(t, x, v(t, x), Dv(t, x), D^2v(t, x)) = \beta v - \inf_u \left\{ k(u, x) + b(u, x) \cdot \partial_x v + \frac{1}{2} \text{Tr}(\sigma(u, x) \sigma^\top(u, x) \partial_{xx}^2 v) \right\} \quad (53)$$

Assuming this PDE has a sufficiently regular solution, then by [2], the 2BSDE associated with our PDE is:

$$dY_t = f(t, X_t, Y_t, Z_t, \Gamma_t)dt + Z_t^\top \circ dX_t \quad (54)$$

$$dZ_t = A_t dt + \Gamma_t dX_t \quad (55)$$

$$Y_T = g(X_T) \quad (56)$$

and is solved by:

$$Y_t = v(t, X_t), \quad Z_t = Dv(t, X_t) \quad (57)$$

$$\Gamma_t = D^2 v(t, X_t), \quad A_t = \mathcal{L}Dv(t, X_t) \quad (58)$$

where $\mathcal{L}$ denotes the Dynkin (infinitesimal) operator without the drift term.

Note that $Z_t^\top \circ dX_t$ denotes the Fisk-Stratonovich integration and

$$Z_t^\top \circ dX_t = Z_t^\top dX_t + \frac{1}{2} \text{Tr}(\Gamma_t \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top) dt \quad (59)$$

Hence the 2BSDE associated with our PDE is:

$$\begin{aligned} dY_t &= f(t, X_t, v(t, X_t), Dv(t, X_t), D^2 v(t, X_t))dt \\ &\quad + Z_t^\top dX_t + \frac{1}{2} \text{Tr}(D^2 v(t, X_t) \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top) dt \end{aligned} \quad (60)$$

$$dZ_t = A_t dt + \Gamma_t dX_t \quad (61)$$

$$Y_T = g(X_T) \quad (62)$$

3.2 Proving 2BSDE (Forward Direction) Using Itô's Formula

Set $Y_t = V(t, X_t)$, $Z_t = V_x(t, X_t)$ where $V(t, x)$ is assumed to be a smooth solution to (51).

Set $dX_t = b(u_t, X_t)dt + \sigma(u_t, X_t)dW_t$

Applying Itô's multidimensional formula from (21) to Y_t , we get:

$$\begin{aligned} dY_t &= dV(t, X_t) = V_t(t, X_t)dt + \nabla V^\top(t, X_t)dX_t + \frac{1}{2} \text{Tr}(H V(t, X_t) \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top)dt \\ &= f(t, X_t, V(t, X_t), DV(t, X_t), D^2 V(t, X_t))dt \\ &\quad + Z_t^\top dX_t + \frac{1}{2} \text{Tr}(D^2 V(t, X_t) \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top)dt \end{aligned} \quad (63)$$

as in (60). Hence Y satisfies the 2BSDE.

Now, applying Itô's formula to Z gives:

$$\begin{aligned} dZ_t &= dV_x(t, X_t) \\ &= \frac{\partial}{\partial t} V_x(t, X_t)dt + \nabla V_x^\top(t, X_t)dX_t + \frac{1}{2} \text{Tr}(H Z_t(t, X_t) \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top)dt \end{aligned} \quad (64)$$

Now, $\mathcal{L}Z_t = \frac{\partial}{\partial t} Z_t + \frac{1}{2} \text{Tr}(D^2 Z_t \sigma(u^{\text{aux}}, X_t) \sigma(u^{\text{aux}}, X_t)^\top)$. So let $A_t := \mathcal{L}Z_t$ and note that Γ_t is symmetric. Then:

$$dZ_t = A_t dt + \Gamma_t dX_t \quad (65)$$

4 Implementation and Examples: Testing Against Analytical Solutions

Consider the following process

$$dX_t = (X_t + u_t)dt + dW_t, \quad X_0 = x \in \mathbb{R}^d, \quad (66)$$

with the objective

$$J(u; x) = \mathbb{E} \left[\int_0^T (3X_t^\top X_t + u_t^\top u_t) dt + X_T^\top X_T \right]. \quad (67)$$

This is a simple example of stochastic Linear Quadratic problem (SLQ problem). The analytic optimal control to this problem is

$$u_t^* = -P(t)X_t, \quad (68)$$

where P solves the matrix Riccati equation

$$-\frac{dP(t)}{dt} = 2P(t) - P(t)P(t) + 3I, \quad P(T) = I. \quad (69)$$

In this simple example, P will just be a diagonal matrix with identical diagonal entries $p(t)$, which solves

$$-\frac{dp(t)}{dt} = 2p(t) - p^2(t) + 3, \quad p(T) = 1. \quad (70)$$

4.1 Analytical Solutions

4.1.1 Optimal Control

Let $q(t) = p(T - t)$ so $p(t) = q(T - t)$. Then applying the chain rule:

$$\begin{aligned} \frac{dq}{dt} &= \frac{d}{dt}p(T - t) = -\frac{dp}{dt}(T - t) \\ &= 2p(T - t) - p^2(T - t) + 3 \\ &= 2q(t) - q^2(t) + 3 \end{aligned} \quad (71)$$

with initial condition $q(0) = p(T) = 1$. Separating variables:

$$\int \frac{1}{2q - q^2 + 3} dq = \int dt \quad (72)$$

Now we factor the denominator and use partial fractions:

$$\begin{aligned} \frac{1}{2q - q^2 + 3} &= \frac{-1}{q^2 - 2q - 3} = \frac{-1}{(q - 3)(q + 1)} \\ &= \frac{A}{q - 3} + \frac{B}{q + 1} \end{aligned} \quad (73)$$

To find the coefficients, we have $-1 = A(q+1) + B(q-3)$. Setting $q = -1$ gives $-1 = -4B$, so $B = \frac{1}{4}$. Setting $q = 3$ gives $-1 = 4A$, so $A = -\frac{1}{4}$. Therefore:

$$\begin{aligned}\frac{1}{4} \int \left(\frac{1}{q+1} - \frac{1}{q-3} \right) dq &= \int dt \\ \frac{1}{4} [\ln(q+1) - \ln(q-3)] &= t + C \\ \frac{1}{4} \ln \left(\frac{q+1}{q-3} \right) &= t + C\end{aligned}\tag{74}$$

Exponentiating both sides and solving for q :

$$\begin{aligned}\frac{q+1}{q-3} &= \hat{A}e^{4t} \\ q+1 &= \hat{A}e^{4t}(q-3) \\ q+1 &= q\hat{A}e^{4t} - 3\hat{A}e^{4t} \\ q(1 - \hat{A}e^{4t}) &= -1 - 3\hat{A}e^{4t} \\ q &= \frac{-1 - 3\hat{A}e^{4t}}{1 - \hat{A}e^{4t}}\end{aligned}\tag{75}$$

Applying the initial condition $q(0) = 1$:

$$1 = \frac{-1 - 3\hat{A}}{1 - \hat{A}} \Rightarrow -1 - 3\hat{A} = 1 - \hat{A} \Rightarrow \hat{A} = -1\tag{76}$$

Therefore, the final solution is:

$$q = \frac{3e^{4t} - 1}{1 + e^{4t}}\tag{77}$$

or in terms of p ,

$$p = \frac{3e^{4(T-t)} - 1}{1 + e^{4(T-t)}}\tag{78}$$

Hence the optimal control to the above problem is

$$u_t^* = -P(t)X_t,\tag{79}$$

where $P(t) = p(t)I$ with p as above.

4.1.2 Value Function

Our *deep 2BSDE* method that we implement solves for $v(0, x_0)$, where the HJB equation is:

$$\frac{\partial v}{\partial t} - \beta v + \inf_u \left\{ k(u, x) + b(u, x) \frac{\partial v}{\partial x} + \frac{1}{2} \sigma^2(u, x) \frac{\partial^2 v}{\partial x^2} \right\} = 0\tag{80}$$

with terminal condition $v(T, x) = g(x)$.

The solution is given by:

$$v(t, x) = \inf_u \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(u_s, X_s) ds + e^{-\beta(T-t)} g(X_T) \right] \quad (81)$$

Therefore:

$$v(0, x_0) = \inf_u \mathbb{E} \left[\int_0^T e^{-\beta s} k(u_s, X_s) ds + e^{-\beta T} g(X_T) \right] \quad (82)$$

The current example has objective function $J(u; x)$ where

$$J(u; x) = \mathbb{E} \left[\int_0^T (3X_s^T X_s + U_s^T U_s) dt + X_T^T X_T \right] \quad (83)$$

Hence, if we set $\beta = 0$, then $v(0, x_0) = \inf_u J(u; x)$ with:

$$k(u_t, X_t) = 3X_t^T X_t + U_t^T U_t \quad \text{and} \quad g(X_T) = X_T^T X_T \quad (84)$$

.

These functions can be used in our existing 2BSDE framework without the need for modification.

Substituting them in, the HJB becomes:

$$\frac{\partial v}{\partial t} + \inf_u \left\{ 3X^T X + U^T U + (u + X) \frac{\partial v}{\partial x} + \frac{1}{2} \frac{\partial^2 v}{\partial x^2} \right\} = 0 \quad (85)$$

Assume the value function $v(t, x)$ takes the analytical form:

$$v(t, x) = x^T p x + q \quad (\text{omitting } t \text{ from } p \text{ and } q) \quad (86)$$

In one dimension, this is $px^2 + q$.

Then:

$$\begin{aligned} \frac{\partial v}{\partial t} &= p'x^2 + q' \\ \frac{\partial v}{\partial x} &= 2px \\ \frac{\partial^2 v}{\partial x^2} &= 2p \end{aligned} \quad (87)$$

Substituting the optimal control $u^* = -px$ (which can be found by differentiating the terms under the infimum and minimising) and the value function ansatz into the HJB gives:

$$p'x^2 + q' + 3x^2 + p^2x^2 + (x - px)2px + p = 0 \quad (88)$$

Rearranging terms:

$$(p' + 3 - p^2 + 2p)x^2 + (q' + p) = 0 \quad (89)$$

The x^2 coefficient is exactly the Riccati equation solved above. Hence by setting the constant term to zero, we have

$$\int_t^T \frac{dq}{dt} dt = - \int_t^T p dt \quad (90)$$

and

$$\int_t^T \frac{dq}{dt} dt = q(T) - q(t) = -q(t) \quad \text{since } x^2 = v(T, x) = p(T)x^2 + q(T) \quad (91)$$

by definition of the terminal condition which has no constant term. Therefore,

$$\begin{aligned} q(t) &= \int_t^T p dt = \int_t^T \frac{3e^{4(T-s)}}{1 + e^{4(T-s)}} ds \\ &= \ln \left(1 + e^{4(T-t)} \right) + t - T - \ln(2) \end{aligned} \quad (92)$$

Thus:

$$v(0, x_0) = p(0)x_0^2 + q(0) \quad (93)$$

For the purposes of this example, let $x_0 = 0$ and $T = 0.1$. Then

$$v(0, x_0) = q(0) = \ln \left(1 + e^{0.4} \right) - 0.1 - \ln(2) \quad (94)$$

4.2 Numerical Results

We implement the *deep 2BSDE* method as described in Section 2. All code for this construction can be found in [A.4](#).

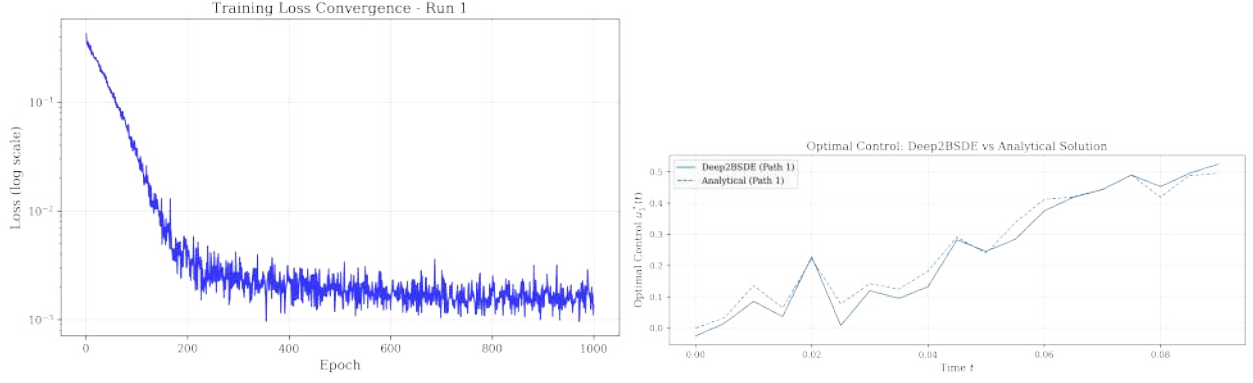
We first define the drift, diffusion and cost functions and set $\beta = 0$ as above.

We then define a function representing the analytical solution of the optimal control derived above, for use later on in plotting a comparison between the numerically approximated optimal control and the true analytical solution. We do the same for the value function.

We then apply the *deep 2BSDE* method to our HJB equation defined earlier.

We can then define functions to run the algorithm a number of times and visualize its output. Here is a simulation of 3 realizations of the algorithm.

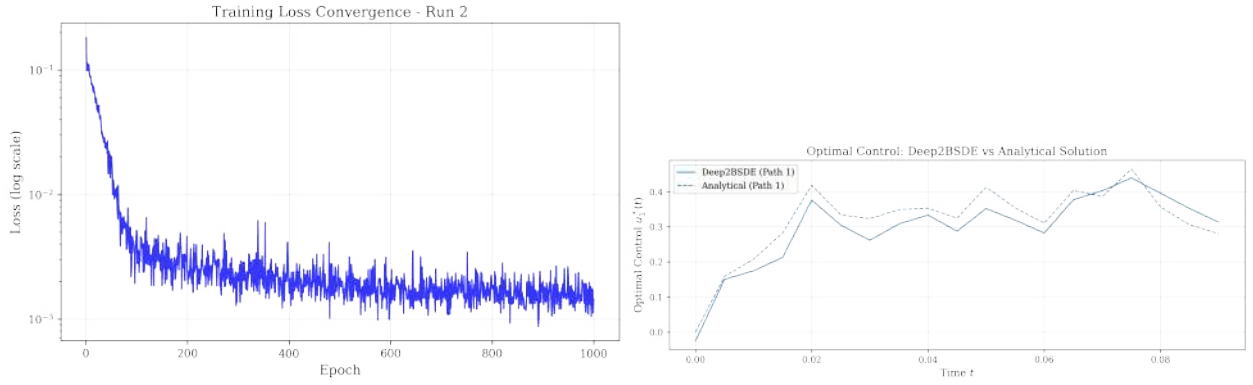
Figure [1](#) summarizes the results for Run 1:



Run 1 Results

Quantity	Value
Analytical solution	0.11986807
Deep2BSDE approximation	0.11886458
Absolute error	1.00e-03
Relative error (%)	0.8372

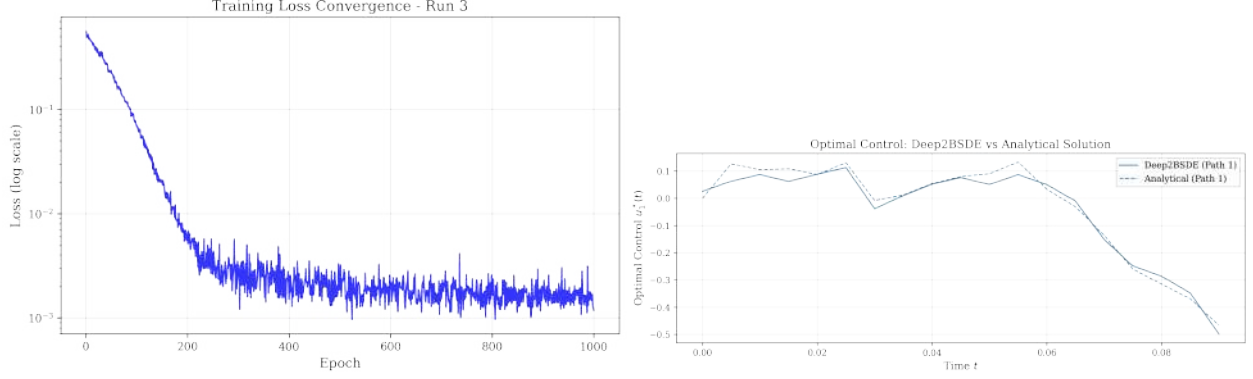
Figure 2 summarizes the results for Run 2:



Run 2 Results

Quantity	Value
Analytical solution	0.11986807
Deep2BSDE approximation	0.12022541
Absolute error	3.57e-04
Relative error (%)	0.2981

Figure 3 summarizes the results for Run 3:



Run 3 Results

Quantity	Value
Analytical solution	0.11986807
Deep2BSDE approximation	0.11903564
Absolute error	8.32e-04
Relative error (%)	0.6945

The summary of all runs is presented in Figure 4:

Summary of All Runs

Quantity	Value
Analytical solution	0.11986807
Mean Deep2BSDE approximation	0.11937521
Absolute error	4.93e-04
Relative error (%)	0.4112

As we can see, the average approximation is extremely close to the true value.

Furthermore, we can see that the optimal control is being found successfully by the algorithm since it maps very well onto the analytic solution.

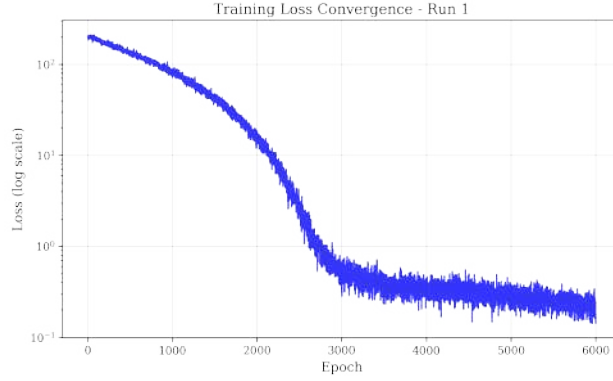
4.3 Investigation

4.3.1 Varying Dimension

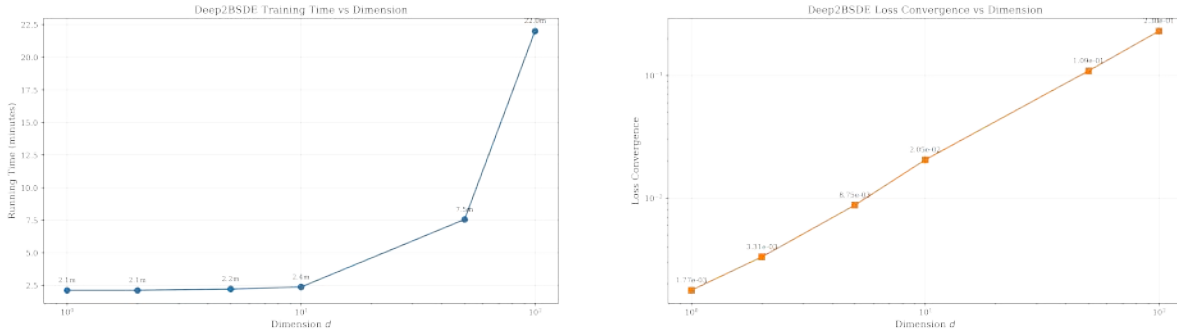
These experiments have been done with $x_0 = 0$, $N = 20$ and with the process X_t in one dimension. We might now wonder how the convergence of the loss is affected by varying the dimension d . We will assess running time using a constant number of 1000 epochs per simulation. We will assess converged loss using however many epochs are necessary to achieve convergence. The following table summarizes the results.

Below is the plot for the loss convergence for $d = 100$ and 6000 epochs. This took 5h 14m 48s to run on a 2.3 GHz 8-core Intel i9.

d	Running Time	Loss Convergence	Notes
1	2m 7s 548ms	1.77×10^{-3}	
2	2m 7s 275ms	3.31×10^{-3}	
5	2m 13s 708ms	8.75×10^{-3}	
10	2m 22s 467ms	2.05×10^{-2}	
50	7m 33s 287ms	1.09×10^{-1}	2000 epochs for convergence
100	22m 6s 979ms	2.30×10^{-1}	6000 epochs for convergence



We can plot the dimension d against both the running time and converged loss to see how they scale based on the scaling of d .



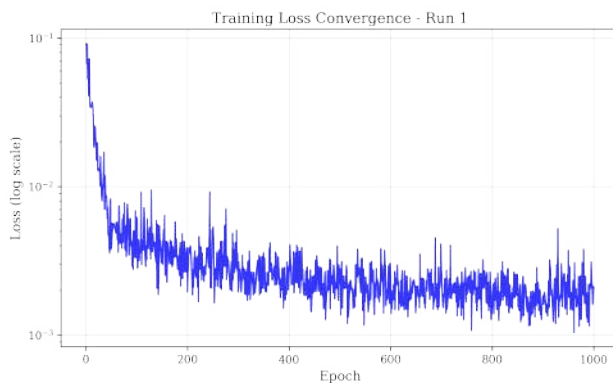
As we can see, the running time scales exponentially and the loss scales linearly.

4.3.2 Varying A Constant Control - Effect on Loss

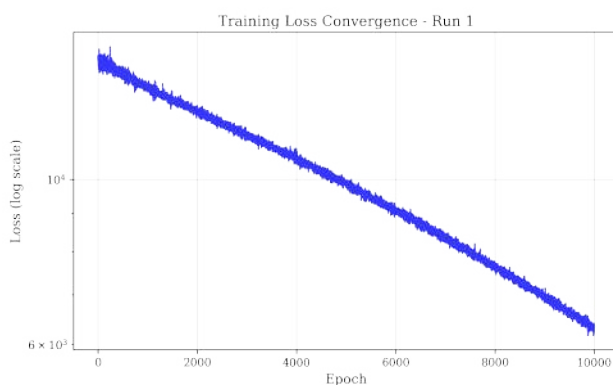
Let us recall Example 1.3.3, where the control u_t was a constant a . One might wonder how a similar situation would affect the current example; we can impose that $u_t \equiv a$, a constant, in the underlying stochastic process. Note, however, that this does not mean we can forget about the infimum in the function f within our HJB equation. While the control will be constant in the stochastic process, it is not constant in our chosen cost function, so we must still minimize over this.

Since the code required minimal modifications, we have omitted this from the appendix. We ran three simulations of the algorithm, each with a different choice of a . The results are as follows: For

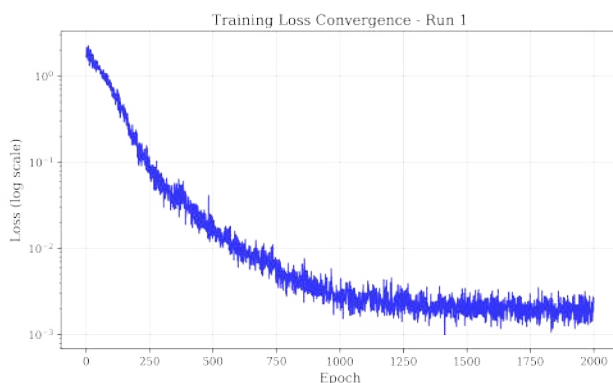
$u_t = 1$, we see the following loss convergence:



If we increase u_t to, say, $u_t = 100$, we see the following loss convergence over 10,000 epochs:



Even over 10,000 epochs, the loss fails to converge and seems to decrease at a constant rate. If we decrease a to 10 and run 2000 epochs, we see the following results:



With $a = 1$ and $a = 10$, the loss converges to approximately 2.01×10^{-3} and 1.97×10^{-3} respectively. In both cases, the algorithm approximated a solution of around 1.32. Due also to the behavior of the loss in the $a = 100$ case, it is reasonable to believe that if the control in the stochastic process is constant, varying it has no effect on the solution of the value function or the

converged loss. Increasing the constant control only causes the algorithm to reach that convergence and solution over a longer duration.

5 Measure-valued Martingale Process

Consider the measure-valued process

$$d\tilde{\xi}_t^{u;\mu} = \tilde{\sigma}(u_t, \tilde{\xi}_t^{u;\mu})dW_t \quad (95)$$

from equation (23) in [4], where $\tilde{\xi}_t^{u;\mu} = (\xi_t^{u;\mu}(\{x_i\}), \dots, \xi_t^{u;\mu}(\{x_N\}))$ and N is such that $\tilde{\xi}^{u;\mu}$ is a Δ^{N-1} (N -dimensional standard simplex) -valued process. Let $\tilde{\xi}_t^{u;\mu}$ be supported on $\{x_1, \dots, x_N\} = \{1, 0, -1\}$. The function $\tilde{\sigma} : \mathcal{U} \times \Delta^{N-1} \rightarrow \mathbb{R}^N$ is given by

$$\tilde{\sigma}(v, \theta) = \left(\theta_1 \left(h(v, x_1) - \sum_{n=1}^N \theta_n h(v, x_n) \right), \dots, \theta_N \left(h(v, x_N) - \sum_{n=1}^N \theta_n h(v, x_n) \right) \right) \quad (96)$$

and the function h is a drift function given by $h(u, x) := ux$. With initial condition $\mu = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ these equations now define a controlled 3-dimensional measure-valued process.

Now, define a cost function as

$$c(\mu, \rho) := \mu(\varphi)^2 + \alpha \mathbb{V}\text{ar}_\mu(\bar{\rho} - \rho) - \frac{1}{\beta} \mathbb{C}\text{ov}_\mu(\varphi, \rho)^2 \quad (97)$$

as in [5], where $\alpha = 1$, $\beta = 1$ and $\bar{\rho} = h(1, \cdot)$. φ can be any continuous function here so for simplicity, we will use the identity. Here we are also using the notation that, for $\mu \in \mathcal{P}_p$ (probability measures whose p th moment is finite) and $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}$,

$$\begin{aligned} \mu(\varphi) &:= \int_{\mathbb{R}^d} \varphi(x) \mu(dx), \\ \mathbb{C}\text{ov}_\mu(\varphi, \psi) &:= \mu(\varphi\psi) - \mu(\varphi)\mu(\psi) \\ \mathbb{V}\text{ar}_\mu(\varphi) &:= \mathbb{C}\text{ov}_\mu(\varphi, \varphi). \end{aligned} \quad (98)$$

Note that these definitions are in the case where the support of the measure-valued martingale is infinite-dimensional. Since we are working in the finite-dimensional case, we understand the definition of $\mu(\varphi)$ to be

$$\mu(\varphi) := \sum_{i=1}^N \varphi(x_i) \mu_i \quad (99)$$

5.1 Integration With *deep 2BSDE*

Note that ρ in the cost function above is not exactly interchangeable with u_t for application in the 2BSDE system derived earlier. In [5], where the cost function is defined, the MVM of interest is the (weak) solution of the following real-valued martingale equation

$$\xi_t(\varphi) = \xi_0(\varphi) + \int_0^t \mathbb{C}\text{ov}_{\xi_s}(\varphi, \rho_s) dW_s \quad (100)$$

which has an infinite-dimensional support. Assume this is now N -dimensional as above. Then in differential notation

$$\begin{aligned}
d\tilde{\xi}_t(\varphi) &= \mathbb{Cov}_{\tilde{\xi}_t}(\varphi, \rho_t) dW_t \\
&= (\tilde{\xi}_t(\varphi \rho_t) - \tilde{\xi}_t(\varphi) \tilde{\xi}_t(\rho_t)) dW_t \\
&= \left(\left(\sum_{i=1}^N \varphi(x_i) \rho_t(x_i) \xi_t(\{x_i\}) \right) - \left(\sum_{i=1}^N \varphi(x_i) \xi_t(\{x_i\}) \right) \left(\sum_{i=1}^N \rho_t(x_i) \xi_t(\{x_i\}) \right) \right) dW_t \quad (101) \\
&= \left(\sum_{i=1}^N \varphi(x_i) \xi_t(\{x_i\}) \left(\rho_t(x_i) - \left(\sum_{j=1}^N \rho_t(x_j) \xi_t(\{x_j\}) \right) \right) \right) dW_t
\end{aligned}$$

That is, as a measure-valued martingale,

$$d\tilde{\xi}_t(\{x_i\}) = \tilde{\xi}_t(\{x_i\}) \left(\rho_t(x_i) - \left(\sum_{j=1}^N \rho_t(x_j) \xi_t(\{x_j\}) \right) \right) dW_t \quad (102)$$

which has the dW_t coefficient of the form of $\tilde{\sigma}(u_t, \tilde{\xi}_t)_i$ where

$$\rho_t(x_i) = h(u_t, x_i). \quad (103)$$

Hence, the cost function we are interested in is actually $k(\tilde{\xi}_t^{u;\mu}, u_t) := c(\tilde{\xi}_t^{u;\mu}, h(u_t, \cdot))$. The first term is

$$\tilde{\xi}_t^{u;\mu}(\varphi)^2 = \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2 \quad (104)$$

and the middle term is

$$\begin{aligned}
\mathbb{Var}_{\tilde{\xi}_t^{u;\mu}}(\bar{\rho} - \rho) &= \tilde{\xi}_t^{u;\mu}((\bar{\rho} - \rho)^2) - \tilde{\xi}_t^{u;\mu}(\bar{\rho} - \rho)^2 \\
&= \tilde{\xi}_t^{u;\mu}((h(1, \cdot) - h(u, \cdot))^2) - \tilde{\xi}_t^{u;\mu}(h(1, \cdot) - h(u, \cdot))^2 \\
&= \tilde{\xi}_t^{u;\mu}((h(1 - u, \cdot))^2) - \tilde{\xi}_t^{u;\mu}(h(1 - u, \cdot))^2 \quad \text{since } h \text{ is linear} \\
&= \sum_{i=1}^N h(1 - u, x_i)^2 \xi_t^{u;\mu}(\{x_i\}) - \left(\sum_{i=1}^N h(1 - u, x_i) \xi_t^{u;\mu}(\{x_i\}) \right)^2 \quad (105) \\
&= \sum_{i=1}^N (1 - u)^2 x_i^2 \xi_t^{u;\mu}(\{x_i\}) - \left(\sum_{i=1}^N (1 - u) x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2
\end{aligned}$$

and the final term is

$$\begin{aligned}
-\frac{1}{\beta} \mathbb{Cov}_\mu(\varphi, \rho)^2 &= -\frac{1}{\beta} \left(\tilde{\xi}_t^{u;\mu}(\varphi h(u, \cdot)) - \tilde{\xi}_t^{u;\mu}(\varphi) \tilde{\xi}_t^{u;\mu}(h(u, \cdot)) \right)^2 \\
&= -\frac{1}{\beta} \left(\left(\sum_{i=1}^N x_i^2 u \xi_t^{u;\mu}(\{x_i\}) \right) - \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right) \left(\sum_{i=1}^N x_i u \xi_t^{u;\mu}(\{x_i\}) \right) \right)^2 \\
&= -\frac{u^2}{\beta} \left(\left(\sum_{i=1}^N x_i^2 \xi_t^{u;\mu}(\{x_i\}) \right) - \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2 \right)^2 \quad (106)
\end{aligned}$$

5.2 Associated Value Function and HJB Equation

The stochastic optimal control problem corresponding to this cost function has a unique analytical solution in infinite time. Namely, the infinite-time value function is given explicitly by

$$v_{\inf}(0, \boldsymbol{\mu}) = \min \mathbb{E} \left[\int_0^\infty e^{-\beta s} k(\tilde{\xi}_s^{u;\mu}, u_s) ds \right] = \frac{1}{\beta} \mu(\varphi)^2 \quad (107)$$

as per [5] Example 10.1.

If, in finite time, we define the objective function by

$$J(t, u; \boldsymbol{\mu}) := \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(\tilde{\xi}_s^{u;\mu}, u_s) ds + e^{-\beta(T-t)} g(\tilde{\xi}_T^{u;\mu}) \right] \quad (108)$$

then the value function is defined as $v_{\text{fin}}(t, \boldsymbol{\mu}) := \min \mathbb{E} [J(t, u; \boldsymbol{\mu})]$ and

$$v_{\text{fin}}(0, \boldsymbol{\mu}) = \min \mathbb{E} \left[\int_0^T e^{-\beta s} k(\tilde{\xi}_s^{u;\mu}, u_s) ds + e^{-\beta T} g(\tilde{\xi}_T^{u;\mu}) \right]. \quad (109)$$

By the Dynamic Programming Principle, the finite-time and infinite-time value functions are exactly equal if $g(x) := v_{\inf}(T, x)$, hence $g(\tilde{\xi}_T^{u;\mu}) := \frac{1}{\beta} \tilde{\xi}_T^{u;\mu}(\varphi)^2$. The choice of T is irrelevant; however, the effects of different values will be investigated.

The HJB equation associated with this value function is

$$\frac{\partial v}{\partial t} - \beta v + \inf_{u \in \mathcal{U}} \left\{ k(\tilde{\xi}_t^{u;\mu}, u) + \frac{1}{2} \text{tr} \left(\sigma(u, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u, \tilde{\xi}_t^{u;\mu}) \frac{\partial^2 v}{\partial x^2} \right) \right\} = 0 \quad (110)$$

and the value function above is the unique viscosity solution of this HJB equation. Knowing the analytical solution, we solve this in the implementation below and verify its correctness.

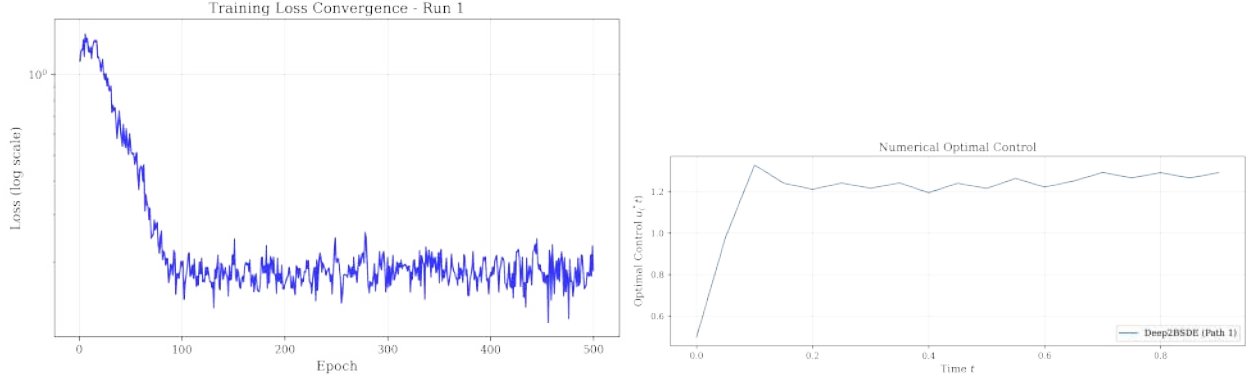
5.3 Implementation and Results

With the cost function above, we can integrate this measure-valued problem into our current algorithm. To adjust our code used in Sections 4 and 5, we first acknowledge that the stochastic process drift term is now zero and the diffusion term is given by the dW_t coefficient in the measure-valued process above.

Also, we notice that in the above process, $\tilde{\xi}_t^{u;\mu}$ and $\tilde{\sigma}$ are 3-dimensional and dW_t is one-dimensional. To implement this within our code, we will treat dW_t as a vector of identical Brownian Increments and the multiplication between $\tilde{\sigma}$ and dW_t as the dot product, satisfying the relationship above. A few more modifications were required to account for this change in dimensionality.

Since the HJB in (107) is constructed in the same way as in Section 4.1.2, our code is already equipped to solve for the value function in (108). However, we also have to impose the constraint that the sum of the elements in the measure equals one and each element is non-negative, which ensures that the measure stays bounded within the simplex domain. Furthermore, we must impose that $\mathcal{Z}$ in (40) is in the tangent space of the simplex, that is, that $\mathcal{A}_n^\theta \cdot \mathbf{1} = 0$ and $\mathcal{G}_n^\theta$ is symmetric and has row sums equal to zero. To see why, recall that in Euclidean space, $\mathcal{Z}$ represents the derivative of the value function. To define a derivative on a 2-simplex, we can, in short, embed the 2-simplex into a ball in $\mathbb{R}^3$ and take the derivative as the directional derivative. This is in the tangent space of the ball in $\mathbb{R}^3$ so when projected back onto the simplex, it should be in the tangent space too.

The following plots are the results of the code in A.5, which implements the system described thus far:



Run 1 Results

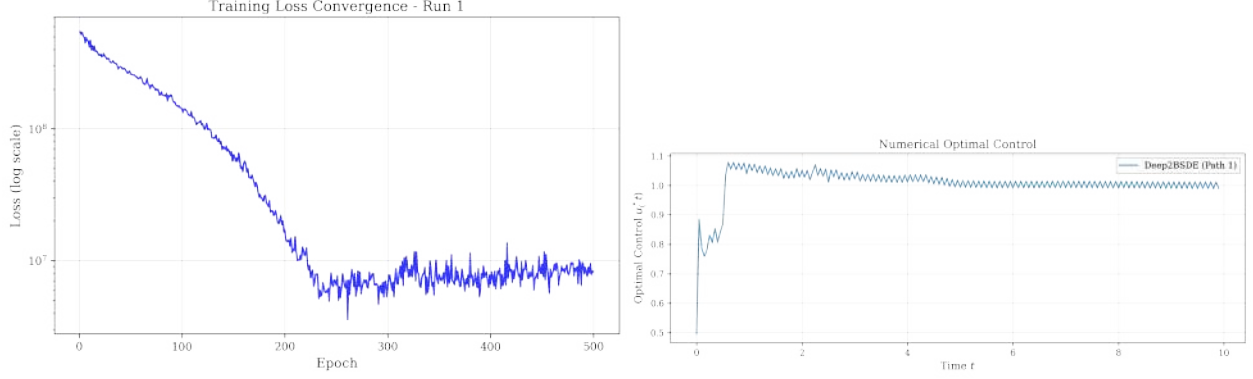
Quantity	Value
Analytical solution	0.00000000
Deep2BSDE approximation	-0.00190689
Absolute error	1.91e-03
Relative error (%)	inf

As we can see, our algorithm converges very close to the true analytic solution, up to some systematic error.

5.4 Investigation

5.4.1 Varying T and N

As mentioned in 5.2, the finite-time formulation has no effect on the solution theoretically, meaning the algorithm's approximation of the solution should be zero for any value of T . Any error is most likely systematic and resulting from the choice of the number/size of discretization steps. To demonstrate this, we can simulate the algorithm with the same step size but different value of T . For example, with $T = 10$ and $N = 200$, we see the following results:

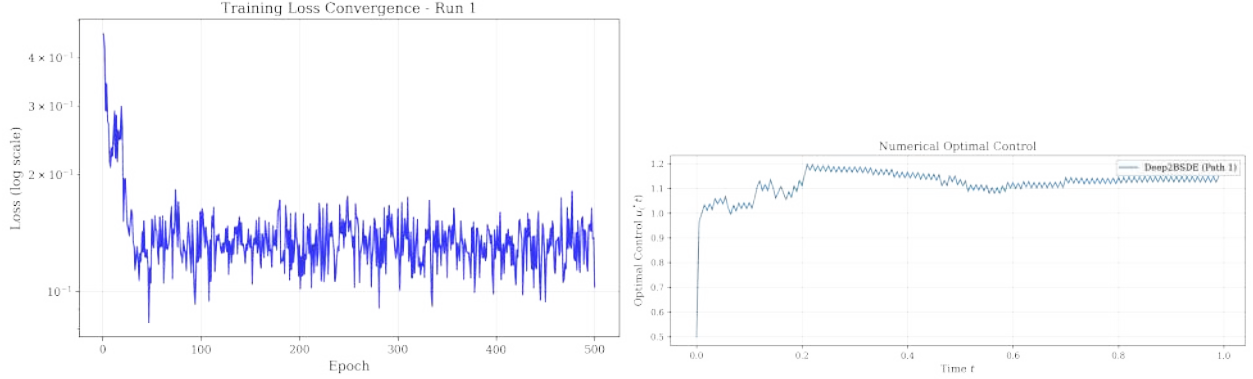


Run 1 Results

Quantity	Value
Analytical solution	0.00000000
Deep2BSDE approximation	-0.02261657
Absolute error	2.26e-02
Relative error (%)	inf

The algorithm still estimates the solution well, supporting the theory that T does not affect the solution. However, notice that over a larger T , the loss converges to a much larger value.

We would expect that, since T does not affect the solution, a smaller step-size would lead to a closer approximation. Here are the results with $T = 1$ and $N = 200$:



Run 1 Results

Quantity	Value
Analytical solution	0.00000000
Deep2BSDE approximation	-0.02247699
Absolute error	2.25e-02
Relative error (%)	inf

The converged loss and approximated solution are very similar to the $N = 20$ case. However, the loss did converge in a lower number of epochs. This indicates that even though the discretization was much finer, there is still some underlying systematic error as mentioned earlier, and the large loss in the $T = 10$ case didn't depend on N . In fact, the systematic error is most likely correlated

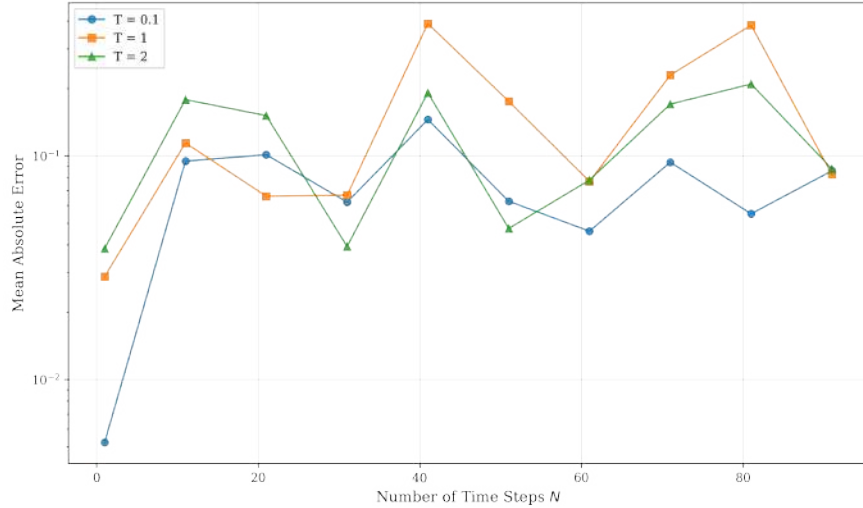
with the number of discretization steps, since the approximation for the $N = 20$ case was closer to the true value than the $N = 200$ case.

5.4.2 Optimizing Input Parameters

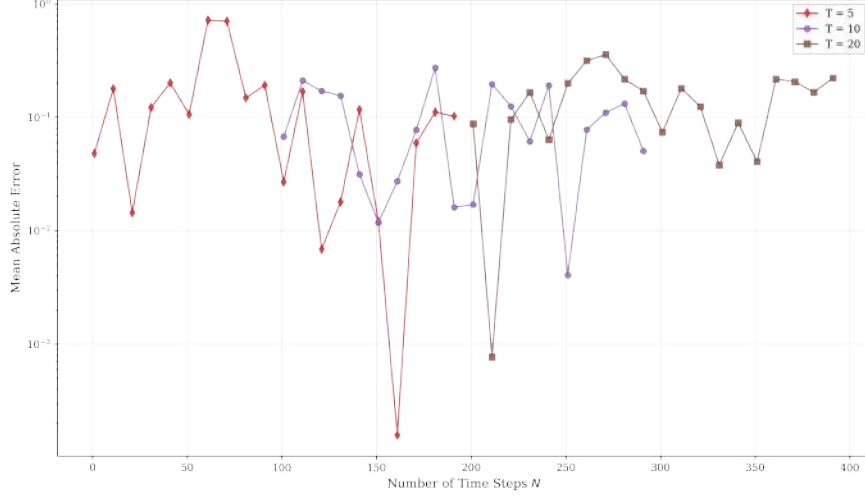
There may be an optimal value of N that provides the lowest systematic error in the approximated solution for each value of T . To perform such an optimization, we need to control all unlearnable variables and initializations apart from N and T . We will set a constant model seed across values of N for a single run.

To keep the stochasticity constant across different discretization sizes, we will generate a master path of Brownian increments for each T and subsample for each value of N . Ideally, the size of this master path would be the LCM of the range of possible N , however this is massive. We will use a 3000-increment master path as values of N will not go above 300. It is important to be aware that, since not every number between 1 and 300 divides 3000, there will be some inconsistencies between total Brownian movements for different N . The largest error is for N such that $3000(\bmod N)$ is largest. This occurs for $N = 273$ in which the misalignment is 9%

For $T = 0.1, 1, 2$, we test the different ratios of $\frac{T}{N}$ by starting with $N = 1$ and incrementing it by 10 each time. For $T = 10, 20$, we start with $N = 101$ and 201 respectively. We perform 3 runs at each N to improve accuracy. The following plots show the mean absolute error against N for each value of T tested:



It appears as though smaller T values benefit from fewer discretization steps in order to reduce absolute error in the solution approximation.



The following table summarizes the values of T and their corresponding values of N and $\frac{T}{N}$ ratios which minimize absolute error.

T	N	T/N
0.1	1	1/10
1	1	1
2	31	1/15
5	161	1/30
10	251	1/25
20	211	1/10

The results are quite varied, which may be applicable to the discretization misalignment. They don't suggest strongly that any ratio of $\frac{T}{N}$ leads to lower systematic error, but a much larger ratio seems to lead to an increase, at least for smaller T .

5.4.3 Loss Explosion

As mentioned in 5.4.1, the larger T value caused loss to converge to a much higher value. This can be explained, somewhat informally.

Within our algorithm, $\mathcal{Y}_n^\theta$ is updated based on the following equation:

$$\begin{aligned}
\mathcal{Y}_{n+1}^\theta = & \mathcal{Y}_n^\theta + \langle \mathcal{Z}_n^\theta, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle \\
& + \left[f(t_n, \mathcal{X}_n, \mathcal{Y}_n^\theta, \mathcal{Z}_n^\theta, \mathcal{G}_n^\theta(\mathcal{X}_n)) + \frac{1}{2} \text{Trace}(\sigma(u^{\text{aux}}, \mathcal{X}_n) \sigma^\top(u^{\text{aux}}, \mathcal{X}_n) \mathcal{G}_n^\theta(\mathcal{X}_n)) \right] (t_{n+1} - t_n)
\end{aligned} \tag{111}$$

where

$$f(t_n, \mathcal{X}_n, \mathcal{Y}_n^\theta, \mathcal{Z}_n^\theta, \mathcal{G}_n^\theta(\mathcal{X}_n)) = \beta \mathcal{Y}_n^\theta - \inf_u \left\{ k(u, \mathcal{X}_n) + b(u, \mathcal{X}_n) \cdot \mathcal{Z}_n^\theta + \frac{1}{2} \text{Tr} \left(\sigma(u, \mathcal{X}_n) \sigma^\top(u, \mathcal{X}_n) \mathcal{G}_n^\theta(\mathcal{X}_n) \right) \right\} \quad (112)$$

Hence, if we assume that terms not involving $\mathcal{Y}_n^\theta$ are bounded tightly enough that they can be assumed constant in comparison to those involving $\mathcal{Y}_n^\theta$, we can write

$$\mathcal{Y}_{n+1}^\theta = \left(1 + \frac{\beta T}{N} \right) \mathcal{Y}_n^\theta + C \quad (113)$$

where C is the remaining terms. This assumption is not unreasonable since the constraints placed on $\mathcal{X}_n, \mathcal{A}_n^\theta$ and $\mathcal{G}_n^\theta$ cause the remaining terms to be small in comparison to $\mathcal{Y}_n^\theta$ throughout the simulation. Note that C depends on β so although this is constant, manipulation involving β must acknowledge this. It may also be possible to bound these remaining terms explicitly and perform a full rigorous error analysis on this simplified formulation of $\mathcal{Y}_n^\theta$. Anyway, this recurrence relation has the following solution, solved by characteristic equation and particular solution:

$$\mathcal{Y}_n^\theta = \left(\mathcal{Y}_0^\theta + \frac{CN}{\beta T} \right) \left(1 + \frac{\beta T}{N} \right)^n - \frac{CN}{\beta T} \quad (114)$$

$$= \mathcal{Y}_0^\theta \left(1 + \frac{\beta T}{N} \right)^n + \frac{CN}{\beta T} \left[\left(1 + \frac{\beta T}{N} \right)^n - 1 \right] \quad (\text{alternative form}) \quad (115)$$

Hence the final value $\mathcal{Y}_N^\theta$ satisfies

$$\mathcal{Y}_N^\theta = \left(\mathcal{Y}_0^\theta + \frac{CN}{\beta T} \right) \left(1 + \frac{\beta T}{N} \right)^N - \frac{CN}{\beta T} \quad (116)$$

As N gets large, this can be approximated by

$$\mathcal{Y}_N^\theta = \left(\mathcal{Y}_0^\theta + \frac{CN}{\beta T} \right) e^{\beta T} - \frac{CN}{\beta T} \quad (117)$$

While an increase in N does cause a linear increase in $\mathcal{Y}_N^\theta$, an increase in T causes an exponential increase. Finally, the terminal condition $g(\mathcal{X}_n^\theta)$ is bounded since $\mathcal{X}_n^\theta$ is bounded, meaning it can't grow as $\mathcal{Y}_n^\theta$ does.

This is why, for $\beta = 1$, we can see the loss convergence becoming so large when T is increased to just 10.

One may now be prompted to explore how manipulation of β , N and T could decrease the converged loss. To do this, we need to write the recursion more specifically and ensure all terms involving β are explicit. Using 111, we can write

$$\mathcal{Y}_{n+1}^\theta \approx \mathcal{Y}_n^\theta + \langle \mathcal{Z}_n^\theta, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle + \left[\beta \mathcal{Y}_n^\theta + \frac{u_{t_n}^2 (B_n - A_n^2)^2}{\beta} - A_n^2 - (1 - u_{t_n})^2 (B_n - A_n^2) \right] \frac{T}{N} \quad (118)$$

where we denote

$$\begin{aligned} A_n &:= \sum_{i=1}^N x_i \xi_{t_n}^{u;\mu}(\{x_i\}) = -\xi_{t_n}^{u;\mu}(\{-1\}) + \xi_{t_n}^{u;\mu}(\{1\}) \\ B_n &:= \sum_{i=1}^N x_i^2 \xi_{t_n}^{u;\mu}(\{x_i\}) = \xi_{t_n}^{u;\mu}(\{-1\}) + \xi_{t_n}^{u;\mu}(\{1\}) \end{aligned} \quad (119)$$

This expression for $\mathcal{Y}_n^\theta$ arises from expanding the cost function and recognizing that the drift term is always zero. Rearranging, we get that

$$\begin{aligned}\mathcal{Y}_{n+1}^\theta &\approx \left(1 + \frac{\beta T}{N}\right) \mathcal{Y}_n^\theta + \frac{u_{t_n}^2 (B_n - A_n^2)^2 T}{\beta N} \\ &\quad + \langle \mathcal{Z}_n^\theta, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle - \frac{A_n^2 + (1 - u_{t_n})^2 (B_n - A_n^2) T}{N}\end{aligned}\tag{120}$$

Now define

$$\begin{aligned}R_n &:= u_{t_n}^2 (B_n - A_n^2)^2 T \\ \tilde{C}_n &:= \langle \mathcal{Z}_n^\theta, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle - \frac{A_n^2 + (1 - u_{t_n})^2 (B_n - A_n^2) T}{N}\end{aligned}\tag{121}$$

then

$$\mathcal{Y}_{n+1}^\theta \approx \left(1 + \frac{\beta T}{N}\right) \mathcal{Y}_n^\theta + \frac{R_n T}{\beta N} + \tilde{C}_n\tag{122}$$

This recursion relation can be solved as before, and thus, for N large,

$$\mathcal{Y}_N^\theta \approx \left(\mathcal{Y}_0^\theta + \frac{\tilde{C}_n N}{\beta T} + \frac{R_n}{\beta^2}\right) e^{\beta T} - \frac{\tilde{C}_n N}{\beta T} - \frac{R_n}{\beta^2}.\tag{123}$$

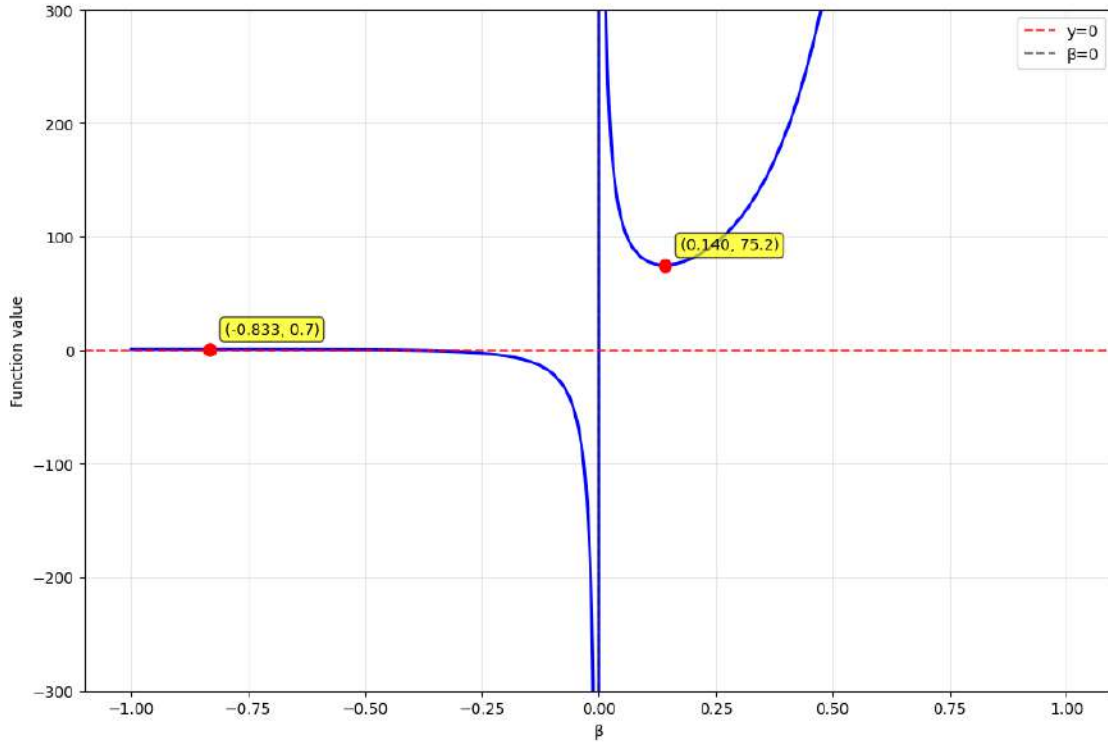
To decrease the converged loss, we would wish for $\mathcal{Y}_N^\theta = g(\mathcal{X}_N) = \frac{1}{\beta} \mathcal{X}_N(\varphi)^2$ which is approximately equal to $\frac{1}{\beta}$. Hence, we would like to either solve or minimize the function

$$\left(\mathcal{Y}_0^\theta + \frac{\tilde{C}_n N}{\beta T} + \frac{R_n}{\beta^2}\right) e^{\beta T} - \frac{\tilde{C}_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}.\tag{124}$$

To see the behavior of this function, we can plot it and visualize the roots. We will assume $\mathcal{Y}_0^\theta = 0$ since by the final epoch, the model should learn this correct analytical solution. First, we must determine the values of $\tilde{C}_n$ and R_n . The following table summarizes the parameter values throughout a realization of the algorithm with $T = 10$ and $N = 20$. Note that, where applicable, these are the means of vectors.

The following is the plot with $\tilde{C}_n$ and R_n equal to 0.1 and 0.5 respectively, as these values are reasonable based on the results in this table. This is effectively setting every element of the vector R_n to 0.5, which, of course, may not be the case in practice. Similarly, the plot shows the mean function value.

Time	$\langle \mathcal{Z}_n^\theta, d\mathcal{X}_n \rangle$	u_{t_n}	A_n	B_n	R_n	$\tilde{C}_n$	$\mathcal{Y}_N^\theta$
0.00	0.0013	0.5067	0.0000	0.6667	0.1712	-0.0798	-3.4644
1.00	-0.0184	0.2279	0.0172	0.6699	0.0348	-0.2182	-3.9280
2.00	0.2104	0.2220	0.0042	0.6602	0.0325	0.0106	-3.9787
3.00	0.0124	0.2843	0.0409	0.6883	0.0555	-0.1643	-2.9614
4.00	-0.0646	0.3372	0.0365	0.6834	0.0776	-0.2151	-1.2295
5.00	0.1834	0.4150	0.0420	0.6894	0.1184	0.0649	3.3013
6.00	0.0772	0.5000	0.0417	0.6962	0.1736	-0.0105	10.5499
7.00	-0.0343	0.5908	0.0458	0.6968	0.2425	-0.0935	20.8534
8.00	0.0195	0.6788	0.0475	0.6969	0.3201	-0.0175	35.7895
9.00	0.1682	0.7697	0.0436	0.6968	0.4117	0.1489	56.3116
9.50	-0.0227	0.8075	0.0401	0.6975	0.4537	-0.0364	68.8218



Note that equating 122 to $\frac{1}{\beta}$ gives the lowest $\beta > 0$ function minimum.

This plot indicates that there exists no positive value of β that is a solution to equation 124. Hence, it is actually not possible to manipulate β , in an acceptable positive range, such that the loss does not explode. It may only be possible to minimize this loss, however, a minimum function value of 75.2 indicates that this decreased loss won't be that close to zero. Furthermore, the value of β that minimizes the function will also change as our estimated parameters $\tilde{C}_n$ and R_n change, so it may be useful to find the exact balance between these parameters that occurs during simulation. The

parameters used above seem to be balanced.

5.4.4 On Early Optimal Control

One may notice from the plots of optimal control thus far that after the first time step (plotted at $t = 0$), the optimal control seems to always be 0.5. This is due to the 'cold-start' - the ADAM optimizer uses momentum and due to the small number of iterations and learning rate employed on the first step, it struggles to get far enough away from the initial value.

To understand its true behavior, we can actually find an analytical solution for the optimal control. Indeed, we wish to minimize

$$k(\tilde{\xi}_t^{u;\mu}, u_t) + \frac{1}{2} \text{tr} \left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u, \tilde{\xi}_t^{u;\mu}) \frac{\partial^2 v}{\partial x^2} \right) \quad (125)$$

since the drift is zero. We have that

$$\begin{aligned} k(\tilde{\xi}_t^{u;\mu}, u_t) &= \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2 \\ &\quad + (1 - u_t)^2 \left(\sum_{i=1}^N x_i^2 \xi_t^{u;\mu}(\{x_i\}) - \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2 \right) \\ &\quad - \frac{u_t^2}{\beta} \left(\left(\sum_{i=1}^N x_i^2 \xi_t^{u;\mu}(\{x_i\}) \right) - \left(\sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) \right)^2 \right)^2. \end{aligned} \quad (126)$$

Denote, as before,

$$\begin{aligned} A_t &:= \sum_{i=1}^N x_i \xi_t^{u;\mu}(\{x_i\}) = -\xi_t^{u;\mu}(\{-1\}) + \xi_t^{u;\mu}(\{1\}) \\ B_t &:= \sum_{i=1}^N x_i^2 \xi_t^{u;\mu}(\{x_i\}) = \xi_t^{u;\mu}(\{-1\}) + \xi_t^{u;\mu}(\{1\}) \\ G_t &:= \frac{\partial^2 v}{\partial x^2}(t, \tilde{\xi}_t^{u;\mu}) \end{aligned} \quad (127)$$

then

$$k(\tilde{\xi}_t^{u;\mu}, u_t) = A_t^2 + (1 - u_t)^2 (B_t - A_t^2) - \frac{u_t^2}{\beta} (B_t - A_t^2)^2. \quad (128)$$

Now,

$$\left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u, \tilde{\xi}_t^{u;\mu}) G_t \right)_{ij} = \sum_{k=1}^N \left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u, \tilde{\xi}_t^{u;\mu}) \right)_{ik} G_{kj} \quad (129)$$

where

$$\left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u, \tilde{\xi}_t^{u;\mu}) \right)_{ij} := \sigma(u_t, \tilde{\xi}_t^{u;\mu})_i \sigma^\top(u, \tilde{\xi}_t^{u;\mu})_j \quad (130)$$

and

$$\begin{aligned}
\sigma(u_t, \tilde{\xi}_t^{u;\mu})_i &= \xi_t^{u;\mu}(\{x_i\}) \left(h(u_t, x_i) - \sum_{n=1}^N \xi_t^{u;\mu}(\{x_n\}) h(u_t, x_n) \right) \\
&= \xi_t^{u;\mu}(\{x_i\}) \left(u_t x_i - \sum_{n=1}^N \xi_t^{u;\mu}(\{x_n\}) u_t x_n \right) \\
&= u_t \xi_t^{u;\mu}(\{x_i\}) (x_i - A_t)
\end{aligned} \tag{131}$$

hence

$$\begin{aligned}
\text{tr} \left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u_t, \tilde{\xi}_t^{u;\mu}) G_t \right) &= \sum_{i=1}^N \sum_{k=1}^N \left(\sigma(u_t, \tilde{\xi}_t^{u;\mu}) \sigma^\top(u_t, \tilde{\xi}_t^{u;\mu}) \right)_{ik} G_{ki} \\
&= \sum_{i=1}^N \sum_{k=1}^N u_t^2 \xi_t^{u;\mu}(\{x_i\}) \xi_t^{u;\mu}(\{x_k\}) (x_i - A_t)(x_k - A_t) G_{ki} \\
&= u_t^2 \sum_{i=1}^N \xi_t^{u;\mu}(\{x_i\}) (x_i - A_t) \sum_{k=1}^N \xi_t^{u;\mu}(\{x_k\}) (x_k - A_t) G_{ki} \\
&= u_t^2 T_t
\end{aligned} \tag{132}$$

where

$$T_t := \sum_{i=1}^N \xi_t^{u;\mu}(\{x_i\}) (x_i - A_t) \sum_{k=1}^N \xi_t^{u;\mu}(\{x_k\}) (x_k - A_t) G_{ki}. \tag{133}$$

So, in fact, we are minimizing

$$A_t^2 + (1 - u_t)^2 (B_t - A_t^2) - \frac{u_t^2}{\beta} (B_t - A_t^2)^2 + \frac{u_t^2 T_t}{2}. \tag{134}$$

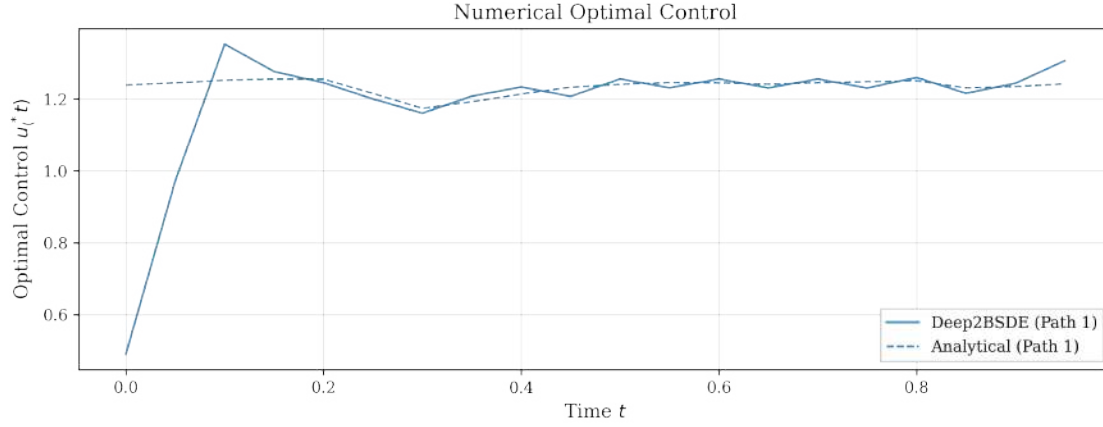
The derivative is

$$\begin{aligned}
&- 2(1 - u_t) (B_t - A_t^2) - \frac{2u_t}{\beta} (B_t - A_t^2)^2 + u_t T_t \\
&= -2 (B_t - A_t^2) + u_t \left(2 (B_t - A_t^2) - \frac{2}{\beta} (B_t - A_t^2)^2 + T_t \right)
\end{aligned} \tag{135}$$

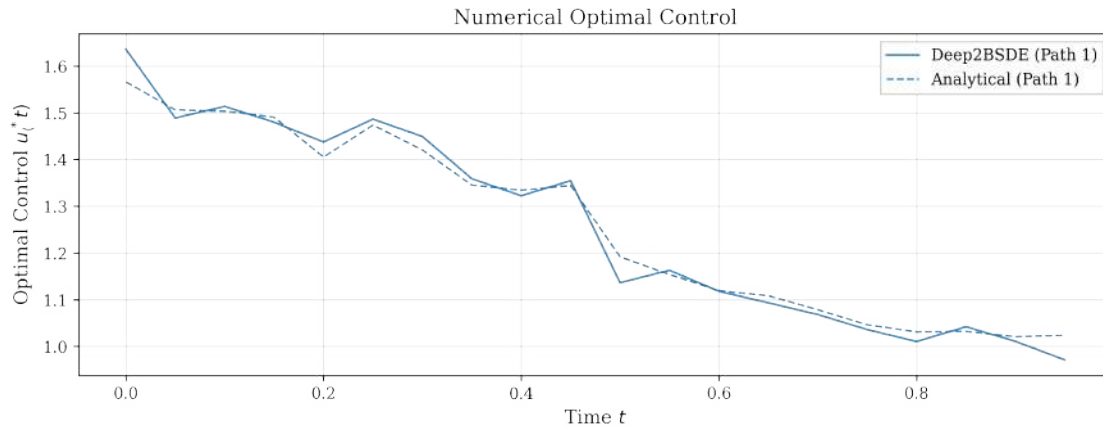
So equating this to zero and solving for u_t gives

$$u_t = \frac{B_t - A_t^2}{B_t - A_t^2 - \frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2}} \tag{136}$$

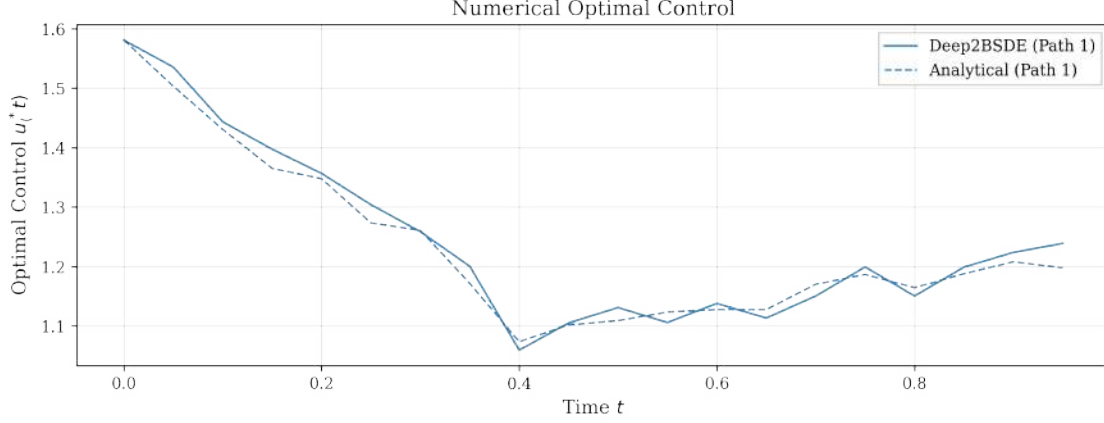
We will assume that this is a minimum. Here is a plot of the numerical optimal control against the analytical solution with $T = 1$, $N = 20$, demonstrating that the optimizer struggles at the start:



An idea to mitigate this early struggle would be to increase the number of iterations on the first optimization step. If we increase from 10 to 50, we see the following plot of optimal control against the analytical solution:

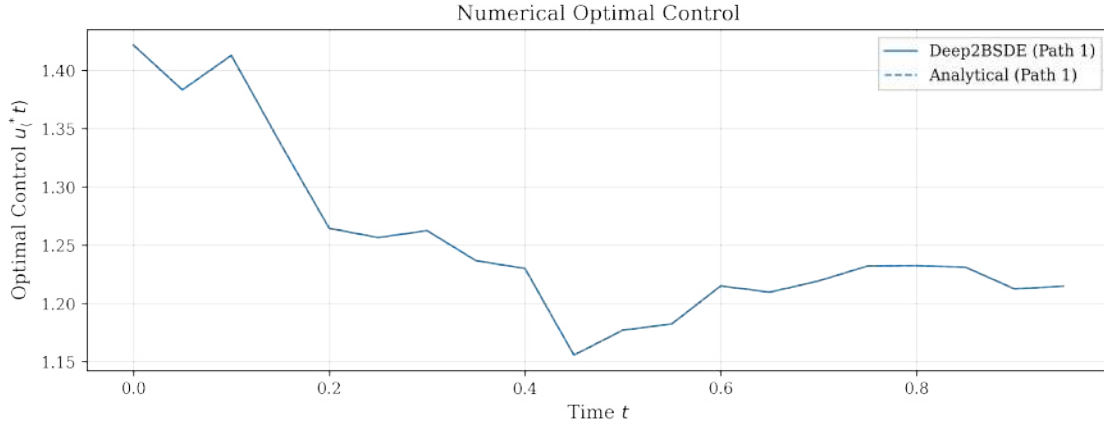


In this plot, it looks as though the optimal control after the first time step is overestimated. This is again due to the momentum that ADAM builds up when optimizing. Since there is only one variable to optimize over, an optimizer like L-BFGS may be more suitable on the first step. Indeed, the plot now looks like this:



Clearly, the first optimal control estimate is much more accurate with this optimizer. Another benefit is that we can assert the maximum number of iterations, with the optimizer being able to stop once it is satisfied that it has reached the global minimum.

This poses the question of whether L-BFGS is more suitable to be used over all time-steps and whether it is more practical, as the automatic stopping could even lead to increases in efficiency. Employing this idea, we see the following plot:



The numerical approximation maps perfectly onto the analytical solution, indicating that the use of this optimizer is a real improvement.

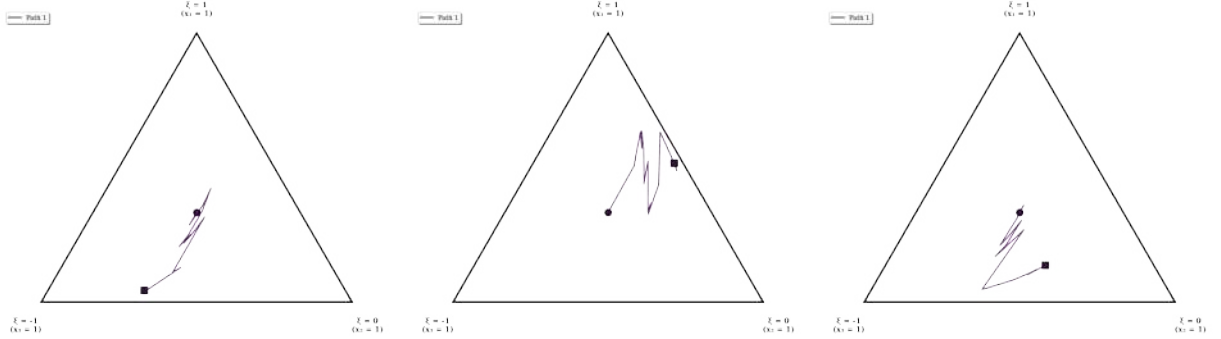
In [5], the theoretical solution for the optimal control is derived. Namely, $u_t = 1$. The issue in the derivation above is that it relies on T_t , which depends on the learnable parameter G_t . Hence, as the neural network learns, we would see that $-\frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2} \rightarrow 0$. The distance of the early optimal control from the theoretical solution results from poor initialization of G_t and slow learning.

5.4.5 Allowing Bound-Absorption in the Simplex

In the construction of the measure-valued process, the constraint that the measure $\tilde{\xi}_t^{u;\mu}$ must stay within the simplex was imposed strictly. That is, an arbitrarily small constant was added to components that would be zero in order to keep them strictly inside the simplex. However, it

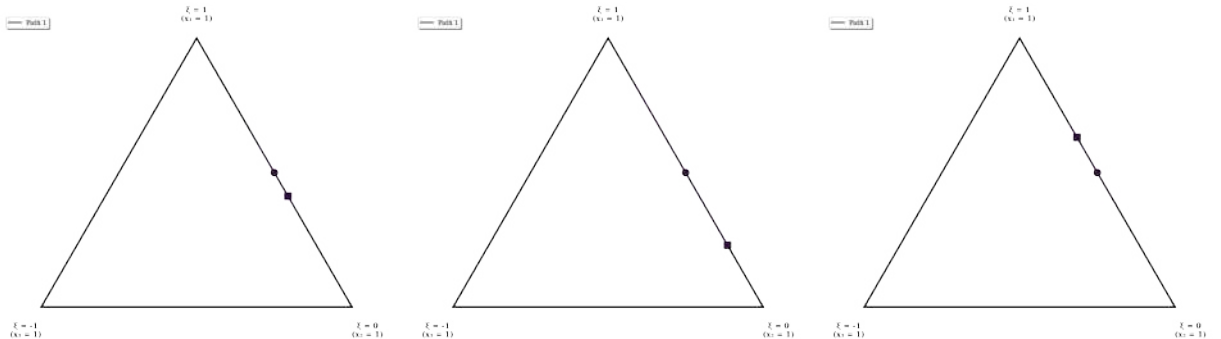
may be interesting to explore how the algorithm behaves when the boundary of the simplex can absorb the process, and that coordinate remains zero for the remainder of the process.

To visualize the different cases, we can modify the code to plot the movement of the process through the simplex. To do this, we can convert the barycentric coordinates to Cartesian coordinates and plot them as usual. Here are three plots of the process moving through the simplex as the model is trained, while imposing the condition that the process stays inside the simplex. The circle denotes the initial condition and the square denotes the final position:



In all three of these simulations, the approximated solution was 0.002. As we can see, the process avoids the boundary anyway, indicating that absorption is irrelevant. Indeed, modifying the code so that the boundaries absorb the process showed that, over three simulations, the process never reached the boundary. Note that this was tested with and without the small perturbation added to the projected values of the process as to eliminate this as a factor affecting the path. However, this was not tested for values of T greater than 1. We may find that the process eventually reaches the boundary, in which case, the algorithm should perform as it does below.

To see the behavior of the algorithm along the boundary, we can instead modify the initial measure $\tilde{\xi}_0^{u;\mu}$ to be on the boundary. Originally, the initial measure was set to $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. By setting it to $(0, \frac{1}{2}, \frac{1}{2})$, we ensure that it starts on the boundary and stays there. Note that the change of initial condition means the analytical solution changes - it now becomes 0.25.



Quantity	Value
Analytical solution	0.25000000
Deep2BSDE approximation	0.25099942
Absolute error	9.99e-04
Relative error (%)	0.3998

Indeed, the algorithm finds the correct solution while staying on the boundary of the simplex.

6 Financial Applications of the *deep 2BSDE* Method

In this section we explore some financially-motivated stochastic optimal control problems, taken from examples in [6].

6.1 The Merton Problem

The Merton problem is a classical portfolio optimization problem (from Merton (1971)), in which an agent seeks to maximize expected (discounted) wealth by trading in a risky asset and risk-free bank account.

Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset S_t and seek to obtain the value function

$$H(S, x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}_{S,x} [U(X_T^\pi)] \quad (137)$$

where $\mathcal{A}_{t,T}$ is the set of (admissible) strategies, S is the asset price and x is the current wealth.

$U(x)$ is the agent's utility function, which outputs their 'satisfaction' with their current wealth. For example, a utility function may encode that higher wealth is more satisfactory, but with diminishing returns such that a constant increase in wealth means a smaller increase in utility as wealth grows.

The price of the risky asset and the wealth at time t are given by the SDEs

$$\begin{aligned} dS_t &= (\mu - r)S_t dt + \sigma S_t dW_t \\ dX_t^\pi &= (\pi_t(\mu - r) + rX_t^\pi)dt + \pi_t \sigma dW_t \end{aligned} \quad (138)$$

with $S_0 = S$ and $X_0^\pi = x$. μ and r represent the (expected) continuously compounded rate of growth of the asset and continuously compounded rate of return of the risk-free bank account respectively. σ represents the *volatility* of the asset.

6.1.1 Implementation with *deep 2BSDE*

Firstly, our implementation solves for an infimum. Hence, the above value function is equivalent to

$$- \inf_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}_{S,x} [-U(X_T^\pi)] \quad (139)$$

and so the value found by our algorithm will be the negative of the above value function.

Also, in practice, the parameters μ and r can be found by estimation using historical data. For simplicity, we will use random, realistic numbers as an example. Assume that $\mu = 0.12$, $r = 0.04$ and $\sigma = 0.18$.

With the setup in section 4.1.2, we have that $\beta = 0$, $k(\pi_t, x) = 0$ and $g(X_t) = -U(X_t)$ which we will define, using *exponential utility*, as

$$U(x) := -e^{-\gamma x} \quad (140)$$

This utility function means that the agent's willingness to take risks does not depend on their wealth level. γ is the agent's risk aversion - in this example we will take $\gamma = 0.5$.

Furthermore, using this utility function means that both the optimal control and value functions have closed-form, analytical solutions which we can use for testing. Namely, by [6], the solutions are

$$\begin{aligned}\pi(x, t) &= \frac{\mu - r}{\gamma \sigma^2} e^{-r(T-t)} \\ v(t, x) &= -e^{-\frac{\mu-r}{2\sigma^2}(T-t) - \gamma x e^{r(T-t)}}\end{aligned}\tag{141}$$

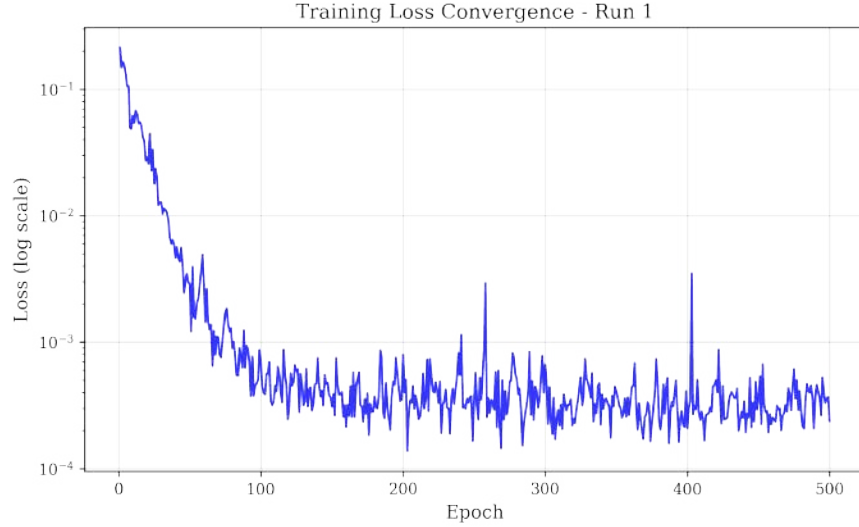
Hence,

$$v(0, x) = -e^{-\frac{\mu-r}{2\sigma^2}T - \gamma x e^{rT}}\tag{142}$$

Now, to encode this problem into our algorithm, we define the following functions:

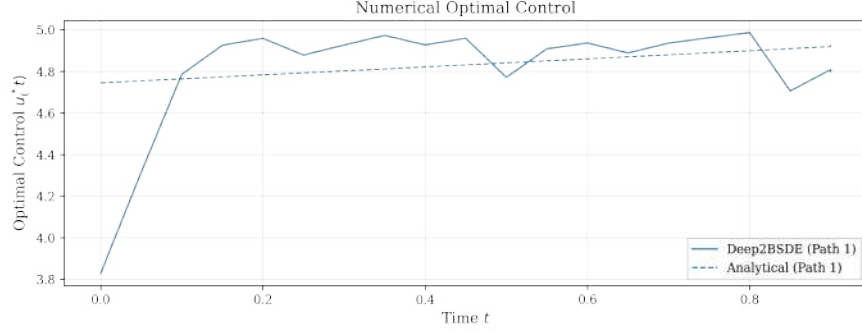
$$\begin{aligned}(\text{drift}) \quad b(u, x) &= (u(\mu - r) + rx) \\ (\text{diffusion}) \quad \sigma(u, x) &= u\sigma\end{aligned}\tag{143}$$

Note that one additional constraint is required; in [6] the analytical optimal control is derived for the maximization problem with the condition that the Hessian matrix of the value function be negative definite. However, our minimization problem means that the solution is actually the negative of the value function that was solved for in the maximization problem. This, in turn, means that minimization requires the condition that the Hessian matrix of our value function be positive definite. With this in mind, we see the following results after running the algorithm, the code for which can be found in A.6:



Run 1 Results

Quantity	Value
Analytical solution	0.53839068
Deep2BSDE approximation	0.53861088
Absolute error	2.20e-04
Relative error (%)	0.0409



Recall that the approximated solution is the negative of the solution to the maximization problem. Here, the analytical solution has also been inverted to display accuracy. The absolute error is extremely small and so is the converged loss, indicating that our algorithm is extremely accurate.

The optimal control solution indicates that it is optimal to invest around \$5 per unit of time, and that the value of the problem is approximately -0.538. This value is quite unintuitive, however, it can be converted back to wealth units, to a value called the *certainty equivalent*. This is the guaranteed wealth that would give the same utility (satisfaction) as the risky strategy. The formula is the inverse of the utility function at the value of the problem, being

$$-\frac{1}{\gamma} \ln(0.538) = -\frac{1}{0.5} \ln(0.538) = 1.24$$

In context, this means that the optimal investment strategy is worth a guaranteed 24% increase in wealth.

Furthermore, the plot of optimal control shows that with the constraint above imposed, the algorithm minimizes correctly. Although the mapping is not perfect, the numerical approximation follows the general trend of the analytical solution of the optimal control. The early optimal control is as in section 5.4.4 - the ADAM optimizer is limited by its learning rate and iteration number. This behavior is even with an increased number of iterations on the first step. In section 5.4.4, it was found that an L-BFGS optimizer was more appropriate, especially for the first time step with no previous control to use as a warm start. When trying to employ that in this example, the optimization actually failed. It may be a sensible next step to understand why this fails and find a way to use L-BFGS to improve the accuracy of the optimal control approximation.

A Appendix - Python Implementation

A.1 Brownian Motion SRW Construction and Plotting

```
import numpy as np
import matplotlib.pyplot as plt

# N is the length of the walk, this is X_j
def rand_jumps(N):
    X = np.random.choice([-1,1], size=N, p=[0.5,0.5])
    return X
```

```
#Sum of X_j - simple random walk
def sym_rand_walk(X):
    M = np.concatenate([[0], np.cumsum(X)])
    return M
```

```
def plot_rw(walk, n):
    # Calculate time steps scaled by n
    steps = np.arange(len(walk)) / n

    plt.figure(figsize=(10, 6))
    plt.plot(steps, walk, 'b-', linewidth=2)
    plt.xlabel('Time')
    plt.ylabel('Position')
    plt.title('Brownian Motion')
    plt.grid(True, alpha=0.3)

    # Set x-axis ticks to integers only
    max_time = steps[-1]
    plt.xticks(np.arange(0, int(max_time) + 1, 1))
    plt.xlim(0, max_time)

    # Set y-axis ticks to integers only
    max_y = int(np.ceil(np.max(walk)))
    min_y = int(np.floor(np.min(walk)))
    plt.yticks(np.arange(min_y, max_y, 1))
    plt.ylim(min_y, max_y)

    ax = plt.gca()
    ax.spines['bottom'].set_position('zero')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    plt.show()
```

```

# Scaled random walk which becomes BM when n --> infinity
def scaled_rand_walk(n,t):
    if isinstance(n*t,int):
        N = n*t
        X = rand_jumps(N)
        W = (1/np.sqrt(n))*sym_rand_walk(X)
        # When nt is not an integer, W found by linear interpolation between
        ↪ nearest points s,t either side of t such that ns,nu are integers
    else:
        N_1 = int(np.floor(n*t))
        N_2 = int(np.ceil(n*t))
        X_1 = rand_jumps(N_1)
        X_2 = rand_jumps(N_2)
        W_1 = (1/np.sqrt(N_1))*sym_rand_walk(X_1)
        W_2 = (1/np.sqrt(N_2))*sym_rand_walk(X_2)
        W = (W_1+W_2)/2
    return W

```

```

n = 100
t = 4
BM = scaled_rand_walk(n,t)
plot_rw(BM, n)

```

A.2 Brownian Motion in Practice

```

import numpy as np
import matplotlib.pyplot as plt

```

```

def Brownian_Increments(M,T):
    W_m = np.zeros(M)
    for i in range(M):
        W_m[i] = np.random.normal(0,np.sqrt(T))
    return W_m

```

```

W_m = Brownian_Increments(5,10)
print(W_m)

```

```

[-0.72652416 -0.78625751  3.20509024 -7.74674169  1.50767567]

```

A.3 Euler Maruyama

A.3.1 Example 1

```

def Euler_Maruyama(N,t):
    W = scaled_rand_walk(N,t)
    Y = np.zeros(N+1)

```

```

Y[0] = 10**(-20)
for n in range(N):
    dW = W[t*(n+1)] - W[t*n]
    Y[n+1] = Y[n] + (1/2)*np.sqrt(Y[n])*dW
    Y[n+1] = max(Y[n+1], 10**(-20))
return Y

```

```

import matplotlib.pyplot as plt
import numpy as np

def plot_euler_maruyama(Y,N,t):

    # Create time points
    time_points = np.linspace(0, t, N+1)

    # Plot the solution
    plt.figure(figsize=(10, 6))
    plt.plot(time_points, Y, 'b-', linewidth=1.5, label='Euler-Maruyama_
→Approximation')
    plt.xlabel('Time t')
    plt.ylabel('X(t)')
    plt.title(f'Numerical Solution')
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()

```

```

N=100000
t=1
EM = Euler_Maruyama(N, t)
plot_euler_maruyama(EM,N,t)

```

A.3.2 Example 2

```

def drift(u,x):
    return u+x

def diffusion(u,x):
    return (1/2)*u - (1/3)*x

def Euler_Maruyama(M,T,N,a):
    #N is the number of time discretisation steps
    #a is a constant control process
    EM = np.zeros((M,N+1))
    for m in range(M):
        W_n = Brownian_Increments(N,T/N)
        for n in range(N):

```

```

        EM[m,n+1] = EM[m,n] + drift(a,EM[m,n])*(T/N) +  $\sqrt{a}$ 
         $\rightarrow$ diffusion(a,EM[m,n])*W_n[n]
    return EM

```

```

M = 5
T = 10
N = 100000
a = 5
X = Euler_Maruyama(M,T,N,a)
print(X)

```

```

[[ 0.00000000e+00 -2.54612592e-02 -6.48598749e-02 ... 2.52164764e+04
 2.52044520e+04 2.51476316e+04]
 [ 0.00000000e+00 5.69085257e-02 3.31971471e-02 ... 3.75348079e+04
 3.76288997e+04 3.74629631e+04]
 [ 0.00000000e+00 -1.57830404e-03 1.95814827e-02 ... 1.29137114e+04
 1.29072579e+04 1.28803992e+04]
 [ 0.00000000e+00 -2.63264038e-03 1.05748572e-02 ... 1.20177029e+05
 1.20514390e+05 1.20456359e+05]
 [ 0.00000000e+00 -8.06349165e-03 -4.78567463e-02 ... 3.30111850e+04
 3.32134158e+04 3.32357138e+04]]

```

```

def plot_first_path(X, T, N):

    # Extract first path
    Y = X[0, :]

    # Create time points: step i is at time i*T/N
    time = np.array([i * T / N for i in range(N + 1)])

    # Simple plot
    plt.plot(time, Y)
    plt.xlabel('time')
    plt.ylabel('X(t)')
    plt.show()

```

```

plot_first_path(X,T,N)

```

A.4 Example Implementation from Section 4

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

```

```
def brownian_training_data(batch_size, N, d, T):

    dt = T / N
    sqrt_dt = np.sqrt(dt)

    # Generate all random numbers at once with correct scale
    training_scenarios = np.random.normal(0, sqrt_dt, (batch_size, N, d))

    return torch.tensor(training_scenarios, dtype=torch.float32)
```

```
def drift(u,x):
    return u+x

def diffusion(u, x):
    batch_size, d = x.shape
    return torch.eye(d, device=x.device).unsqueeze(0).repeat(batch_size, 1, ↵
↵1)
```

```
Beta = 0

def cost_tensor(u, x, t):
    return 3 * torch.sum(x * x, dim=-1, keepdim=True) + torch.sum(u * u, ↵
↵dim=-1, keepdim=True)
```

```
# FOR BETA = 0 !!

def analytical_solution_optimal(T,t,X):
    return -(3*np.exp(4*(T-t))-1)/(1+np.exp(4*(T-t)))*X

def analytical_solution_value(T):
    return np.log(1+np.exp(4*T))-T-np.log(2)
```

```
def numerical_optimal_control(t, X, Z, Gamma, num_iterations=10, lr=0.05, ↵
↵u_init=None):

    X_detached = X.detach()
    Z_detached = Z.detach()
    Gamma_detached = Gamma.detach()

    batch_size, d = X_detached.shape

    # Warm start from previous solution if provided
    if u_init is not None:
        u = u_init.detach().clone().requires_grad_(True)
    else:
        u = torch.zeros_like(X_detached, requires_grad=True)
```

```

# Use Adam optimizer for better convergence
optimizer = torch.optim.Adam([u], lr=lr)

for _ in range(num_iterations):
    optimizer.zero_grad()

    # Compute objective function
    cost_term = cost_tensor(u, X_detached, t) # (batch_size, 1)

    # Drift term:  $b(u, x) \cdot Z$ 
    drift_vals = drift(u, X_detached) # (batch_size, d)
    drift_term = torch.sum(drift_vals * Z_detached, dim=-1,
→keepdim=True) # (batch_size, 1)

    # Diffusion term:  $(1/2) \sigma \sigma^T(u, x) \text{trace}(\Gamma)$ 
    sigma = diffusion(u, X_detached) # (batch_size, d)
    sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
    trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T,
→Gamma_detached).unsqueeze(-1)
    diffusion_term = 0.5 * trace_vals

    # Total objective to minimize
    objective = cost_term + drift_term + diffusion_term
    loss = torch.mean(objective) # Average over batch
    loss.backward()
    optimizer.step()

    return u.detach()

# Fully batched f_func
def f_func_batched(t, X, Y, Z, Gamma, u_optimal):

    # Compute objective function
    cost_term = cost_tensor(u_optimal, X, t) # (batch_size, 1)

    # Drift term:  $b(u, x) \cdot Z$ 
    drift_vals = drift(u_optimal, X) # (batch_size, d)
    drift_term = torch.sum(drift_vals * Z, dim=-1, keepdim=True) #
→(batch_size, 1)

    # Diffusion term:  $(1/2) \sigma \sigma^T(u, x) \text{trace}(\Gamma)$ 
    sigma = diffusion(u_optimal, X) # (batch_size, d)
    sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
    trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T, Gamma).
→unsqueeze(-1)
    diffusion_term = 0.5 * trace_vals

```

```

    # Total objective to minimize
    objective = cost_term + drift_term + diffusion_term

    return Beta*Y - objective

# Time-aware neural networks
class GNet(nn.Module):
    # dimension of hidden layer is 64, not d, for better learning
    def __init__(self, d, hidden_dim=64):
        super(GNet, self).__init__()
        self.d = d
        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim), # +1 for time
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d*d)
        )

    def forward(self, x, t_normalized):
        batch_size = x.shape[0]
        # Concatenate space and time
        x_with_time = torch.cat([x, t_normalized], dim=1)
        output = self.net(x_with_time)
        return output.view(batch_size, self.d, self.d)

class ANet(nn.Module):
    # dimension of hidden layer is 64, not d, for better learning
    def __init__(self, d, hidden_dim=64):
        super(ANet, self).__init__()
        self.d = d
        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim), # +1 for time
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d)
        )

    def forward(self, x, t_normalized):
        x_with_time = torch.cat([x, t_normalized], dim=1)
        return self.net(x_with_time)

class Deep2BSDE(nn.Module):

    def __init__(self, d, T, N, xi, g_func):

```

```

super(Deep2BSDE, self).__init__()
self.d = d
self.T = T
self.N = N
self.h = T / N
self.xi = xi
self.g = g_func

# Learnable initial values - reduced standard deviation
self.Y0 = nn.Parameter(torch.randn(1))
self.Z0 = nn.Parameter(torch.randn(1, d) * 0.1)

self.g_net = GNet(d)
self.a_net = ANet(d)

# Initial strategy parameters - reduced standard deviation
self.g0 = nn.Parameter(torch.randn(d, d) * 0.1)
self.a0 = nn.Parameter(torch.randn(1, d) * 0.1)

self.store_trajectory = False # Flag to enable/disable storage
self.numerical_controls = []
self.analytical_controls = []
self.time_points = []

def forward(self, training_scenarios):

    batch_size = training_scenarios.shape[0]
    device = training_scenarios.device

    # Initialize variables
    Y = self.Y0.expand(batch_size, 1)
    Z = self.Z0.expand(batch_size, -1)
    X = self.xi.expand(batch_size, -1)

    # Pre-compute time values to avoid repeated calculations
    time_steps = torch.arange(self.N, device=device, dtype=torch.
↪float32) * self.h

    u_prev = None

    # Clear previous optimal trajectories if storing
    if self.store_trajectory:
        self.numerical_controls = []
        self.analytical_controls = []
        self.time_points = []

    # Main simulation loop

```

```

for n in range(self.N - 1):
    dW = training_scenarios[:, n, :]

    # Get strategy for this time step
    if n == 0:
        G = self.g0.expand(batch_size, -1, -1)
        A = self.a0.expand(batch_size, -1)
    else:
        t_normalized = torch.full((batch_size, 1), n / self.N,
→device=device)

        G = self.g_net(X, t_normalized)
        A = self.a_net(X, t_normalized)

    u_optimal = numerical_optimal_control(
        time_steps[n], X, Z, G,
        num_iterations=10, lr=0.05,
        u_init=u_prev # Warm start from previous timestep
    )

    if self.store_trajectory:
        self.numerical_controls.append(u_optimal.clone().detach())
        self.analytical_controls.
→append(analytical_solution_optimal(self.T, n*self.h, X).clone().detach())
        self.time_points.append(time_steps[n].item())

    # Update equations
    sigma = diffusion(u_optimal, X)

    #find next X
    X_new = X + drift(u_optimal, X) * self.h + torch.bmm(sigma, dW.
→unsqueeze(-1)).squeeze(-1)
    dX = X_new - X

    #f using prev X for Y update
    f_val = f_func_batched(time_steps[n], X, Y, Z, G, u_optimal)

    #trace term using prev X for Y update
    sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
    trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
→unsqueeze(-1)

    Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
→trace_term) * self.h
    Z = Z + A * self.h + torch.bmm(G, dX.unsqueeze(-1)).squeeze(-1)
    X = X_new

    # Store for next iteration warm start

```

```

        u_prev = u_optimal

    # Final time step
    dW = training_scenarios[:, -1, :]
    if self.N > 1:
        t_normalized = torch.ones((batch_size, 1), device=device) # t = 1
    else:
        G = self.g_net(X, t_normalized)
        A = self.a_net(X, t_normalized)
    else:
        G = self.g0.expand(batch_size, -1, -1)
        A = self.a0.expand(batch_size, -1)

    # Final optimal control with warm start
    u_optimal = numerical_optimal_control(
        time_steps[-1], X, Z, G,
        num_iterations=5, lr=0.1,
        u_init=u_prev
    )

    # Update equations
    sigma = diffusion(u_optimal, X)

    #find next X
    X_new = X + drift(u_optimal, X) * self.h + torch.bmm(sigma, dW.
    ↪unsqueeze(-1)).squeeze(-1)
    dX = X_new - X

    #f using prev X for Y update
    f_val = f_func_batched(time_steps[-1], X, Y, Z, G, u_optimal)

    #trace term using prev X for Y update
    sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
    trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
    ↪unsqueeze(-1)

    Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
    ↪trace_term) * self.h
    X = X_new

    return Y, X

def loss(self, training_scenarios):
    Y_terminal, X_terminal = self.forward(training_scenarios)
    target = self.g(X_terminal)
    return torch.mean((Y_terminal - target)**2)

```

```

def set_research_style():
    plt.rcParams.update({
        'font.family': 'serif',
        'font.size': 11,
        'axes.labelsize': 12,
        'axes.titlesize': 13,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
        'legend.fontsize': 10,
        'axes.linewidth': 0.8,
        'axes.grid': True,
        'grid.alpha': 0.3,
        'lines.linewidth': 1.5,
        'figure.figsize': (8, 6),
        'figure.dpi': 300,
        'savefig.transparent': True,
        'figure.facecolor': 'none'
    })

def train_deep2bsde(model, num_epochs=2000, batch_size=64, lr=0.001):

    # Use Adam with weight decay for better convergence
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)

    # Learning rate scheduler
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
    ↪patience=300, factor=0.8)

    best_loss = float('inf')
    loss_history = []

    for epoch in range(num_epochs):
        training_scenarios = brownian_training_data(batch_size, model.N,
    ↪model.d, model.T)

        # Training step
        optimizer.zero_grad()
        loss = model.loss(training_scenarios)
        loss.backward()

        # Gradient clipping for stability
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        optimizer.step()
        scheduler.step(loss)

        loss_history.append(loss.item())

```

```

        # Track best loss
        if loss.item() < best_loss:
            best_loss = loss.item()

        percentage = (epoch + 1) / num_epochs * 100
        print(f"\rProgress: {percentage:5.1f}%", end='', flush=True)

    print()

    return model, loss_history

def plot_loss_history(loss_history, run_number):
    """Plot loss history for a single run."""
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 5), facecolor='none')
    ax.patch.set_facecolor('none')

    epochs = np.arange(1, len(loss_history) + 1)
    ax.semilogy(epochs, loss_history, 'b-', linewidth=1.5, alpha=0.8)

    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss (log scale)')
    ax.set_title(f'Training Loss Convergence - Run {run_number}')
    ax.grid(True, alpha=0.3)

    print(f"{np.mean(loss_history[-100:]):.2e}")

    plt.tight_layout()
    plt.show()

def create_results_table(analytical_val, numerical_val, title="Results"):
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Deep2BSDE approximation', f'{numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
    ]

```

```

        ['Relative error (%)', f'{rel_error:.4f}']
    ]

    table = ax.table(cellText=data,
                     colLabels=['Quantity', 'Value'],
                     cellLoc='left',
                     loc='center',
                     colWidths=[0.6, 0.4])

    table.auto_set_font_size(False)
    table.set_fontsize(11)
    table.scale(1, 2)

    # Style the table
    for i in range(len(data) + 1):
        for j in range(2):
            cell = table[(i, j)]
            if i == 0: # Header
                cell.set_facecolor('#E6E6FA')
                cell.set_text_props(weight='bold')
            else:
                cell.set_facecolor('#F8F8FF')
                cell.set_edgecolor('black')
                cell.set_linewidth(0.5)

    plt.title(title, fontsize=13, fontweight='bold', pad=20)
    plt.tight_layout()
    plt.show()

def create_summary_table(analytical_val, mean_numerical_val, title="Summary_
↳Results"):
    """Create summary table with mean results."""
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(mean_numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Mean Deep2BSDE approximation', f'{mean_numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
        ['Relative error (%)', f'{rel_error:.4f}']
    ]

```

```

]

table = ax.table(cellText=data,
                  colLabels=['Quantity', 'Value'],
                  cellLoc='left',
                  loc='center',
                  colWidths=[0.6, 0.4])

table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 2)

# Style the table
for i in range(len(data) + 1):
    for j in range(2):
        cell = table[(i, j)]
        if i == 0: # Header
            cell.set_facecolor('#E6E6FA')
            cell.set_text_props(weight='bold')
        else:
            cell.set_facecolor('#F8F8FF')
            cell.set_edgecolor('black')
            cell.set_linewidth(0.5)

plt.title(title, fontsize=13, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

```

```

def plot_control_vs_time(model, num_paths=3):
    set_research_style()

    model.store_trajectory = True
    scenarios = brownian_training_data(num_paths, model.N, model.d, model.T)
    model.forward(scenarios)
    model.store_trajectory = False

    fig, axes = plt.subplots(model.d, 1, figsize=(10, 4 * model.d),
    ↪facecolor='none')
    if model.d == 1:
        axes = [axes]

    for ax in axes:
        ax.patch.set_facecolor('none')

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c']

    for dim in range(model.d):

```

```

        ax = axes[dim]

        for path in range(num_paths):
            color = colors[path % len(colors)]
            numerical_u = [controls[path, dim].item() for controls in model.
↪numerical_controls]
            analytical_u = [controls[path, dim].item() for controls in model.
↪analytical_controls]

            ax.plot(model.time_points, numerical_u, '-', color=color,
                    label=f'Deep2BSDE (Path {path + 1})', linewidth=1.5,
↪alpha=0.8)
            ax.plot(model.time_points, analytical_u, '--', color=color,
                    label=f'Analytical (Path {path + 1})', linewidth=1.2,
↪alpha=0.9)

            ax.set_xlabel('Time $t$')
            ax.set_ylabel(f'Optimal Control $u^*_{{dim + 1}}(t)$')
            ax.set_title('Optimal Control: Deep2BSDE vs Analytical Solution')
            ax.legend(frameon=True, fancybox=False, shadow=False, framealpha=0.9)
            ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

```

def test_multiple_runs(num_runs=5, d=1, T=0.1, N=20, num_epochs=2000):
    """Run multiple experiments with individual results and summary."""
    xi = torch.zeros(1, d)

    def g_func(x):
        return torch.sum(x * x, dim=-1, keepdim=True)

    final_values = []
    analytical_val = analytical_solution_value(T)

    print(f"Starting Deep2BSDE experiments: {num_runs} runs, {num_epochs}
↪epochs each")
    print("=" * 60)

    # Run each experiment
    for run in range(num_runs):

        model = Deep2BSDE(d, T, N, xi, g_func)

```

```

        trained_model, loss_history = train_deep2bsde(model,
↳num_epochs=num_epochs, batch_size=64, lr=0.005)

        final_Y0 = trained_model.Y0.item()
        final_values.append(final_Y0)

        # Plot loss for this run
        plot_loss_history(loss_history, run + 1)

        # Show results table for this run
        create_results_table(analytical_val, final_Y0, f"Run {run + 1}"
↳Results")

        plot_control_vs_time(trained_model, 1)

        # Summary table with mean results
        mean_numerical_val = np.mean(final_values)
        create_summary_table(analytical_val, mean_numerical_val, "Summary of All
↳Runs")

    if __name__ == "__main__":
        test_multiple_runs(num_runs=3, d=1, num_epochs=1000)

```

A.5 3-dimension Measure-valued *deep 2BSDE* Method

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

def project_to_simplex_efficient(x, epsilon=1e-8):
    """Efficient vectorized Euclidean projection"""
    batch_size, d = x.shape
    device = x.device

    # Sort each row in descending order
    x_sorted, _ = torch.sort(x, dim=1, descending=True)

    # Compute cumulative sums
    cumsum = torch.cumsum(x_sorted, dim=1)

    # Compute thresholds for all possible j values
    j_vals = torch.arange(1, d + 1, dtype=x.dtype, device=device).
↳unsqueeze(0)

```

```

thresholds = x_sorted + (1 - cumsum) / j_vals

# Find the largest j for each sample where threshold > 0
valid_mask = thresholds > 0

# For each row, find the last True position
j_indices = torch.zeros(batch_size, dtype=torch.long, device=device)
for i in range(batch_size):
    valid_positions = torch.where(valid_mask[i])[0]
    if len(valid_positions) > 0:
        j_indices[i] = valid_positions[-1]

# Compute tau for each sample
tau = torch.zeros(batch_size, device=device)
for i in range(batch_size):
    j = j_indices[i].item()
    tau[i] = (1 - cumsum[i, j]) / (j + 1)

# Project
projected = torch.clamp(x + tau.unsqueeze(1), min=epsilon)

# Normalize for numerical stability
projected = projected / projected.sum(dim=1, keepdim=True)

return projected

```

```

def brownian_training_data(batch_size, N, d, T):

    dt = T / N
    sqrt_dt = np.sqrt(dt)

    # Generate all random numbers at once with correct scale
    training_scenarios = np.random.normal(0, sqrt_dt, (batch_size, N, 1))

    return torch.tensor(training_scenarios, dtype=torch.float32).
    ↪ expand(batch_size, N, d)

```

```

support = torch.tensor([1.0, 0.0, -1.0])

def drift(u, x):
    return x*0

def diffusion(v, theta):
    d = theta.shape[1]

    support_tensor = support.clone().detach().to(device=theta.device,
    ↪ dtype=theta.dtype)

```

```

if v.shape[1] == 1:
    v_broadcast = v.expand(-1,d)  # (batch_size, 3)
else:
    v_broadcast = v

    # h_vals[b,i] = v[b] * support[i] (broadcasted)
h_vals = v_broadcast * support_tensor  # (batch_size, 3)

    # Weighted sum
weighted_avg = torch.sum(theta * h_vals, dim=1, keepdim=True)  #
→ (batch_size, 1)
    # Broadcast weighted_avg back to (batch_size, 3) for subtraction
weighted_avg_broadcast = weighted_avg.expand(-1, d)
return theta * (h_vals - weighted_avg_broadcast)

```

```

Beta = 1

def cost_tensor(xi, u):

    xi_sum = torch.sum(xi * support, dim=1)  # (batch_size,)
    first_term = xi_sum**2

    support_squared = support**2  # [1, 0, 1]
    support_times_xi = torch.sum(support * xi, dim=1)  # (batch_size,)
    support_squared_times_xi = torch.sum(support_squared * xi, dim=1)  #
→ (batch_size,)

    second_term = ((1-u)**2 * support_squared_times_xi -
                    ((1-u) * support_times_xi)**2)

    # Third term: (-1/Beta) * (covariance term)2
    u_support_squared_xi = u * support_squared_times_xi
    u_support_xi = u * support_times_xi
    covar_term = u_support_squared_xi - support_times_xi * u_support_xi

    final_term = (-1/Beta) * covar_term**2

    result = first_term + second_term + final_term
    return result.unsqueeze(-1)  # (batch_size, 1)

```

```

def numerical_optimal_control(t, X, Z, Gamma, num_iterations=20, lr=0.01,
→ u_init=None):

    X_detached = X.detach()
    Z_detached = Z.detach()
    Gamma_detached = Gamma.detach()

```

```

batch_size, d = X_detached.shape

# Warm start from previous solution if provided
if u_init is not None:
    u = u_init.detach().clone().requires_grad_(True)
else:
    u = torch.zeros(batch_size, 1, requires_grad=True)

# Use Adam optimizer for better convergence
optimizer = torch.optim.Adam([u], lr=lr, eps=1e-8)

for _ in range(num_iterations):
    optimizer.zero_grad()

    # Compute objective function
    cost_term = cost_tensor(X_detached, u) # (batch_size, 1)

    # Drift term:  $b(u, x) \cdot Z$ 
    drift_vals = drift(u, X_detached) # (batch_size, d)
    drift_term = torch.sum(drift_vals * Z_detached, dim=-1,
    ↪keepdim=True) # (batch_size, 1)

    # Diffusion term:  $(1/2) * \text{trace}(\sigma \sigma^T(u, x) \Gamma)$ 
    sigma = diffusion(u, X_detached) # (batch_size, d)
    sigma_sigma_T = torch.einsum('bi,bj->bij', sigma, sigma)
    trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T,
    ↪Gamma_detached).unsqueeze(-1)
    diffusion_term = 0.5 * trace_vals

    # Total objective to minimize
    objective = cost_term + drift_term + diffusion_term
    loss = torch.mean(objective) # Average over batch
    loss.backward()
    torch.nn.utils.clip_grad_norm_([u], max_norm=1.0)
    optimizer.step()

    return u.detach()

# Fully batched f_func
def f_func_batched(t, X, Y, Z, Gamma, u_optimal):

    # Compute objective function
    cost_term = cost_tensor(X, u_optimal) # (batch_size, 1)

    # Drift term:  $b(u, x) \cdot Z$ 
    drift_vals = drift(u_optimal, X) # (batch_size, d)

```

```

        drift_term = torch.sum(drift_vals * Z, dim=-1, keepdim=True) #␣
→(batch_size, 1)

        # Diffusion term: (1/2)*trace(σσT(u,x)Gamma)
        sigma = diffusion(u_optimal, X) # (batch_size, d)
        sigma_sigma_T = torch.einsum('bi,bj->bij', sigma, sigma)
        trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T, Gamma).
→unsqueeze(-1)
        diffusion_term = 0.5 * trace_vals

        # Total objective to minimize
        objective = cost_term + drift_term + diffusion_term

        return Beta*Y - objective

# Time-aware neural networks
class GNet(nn.Module):
    def __init__(self, d, hidden_dim=64):
        super(GNet, self).__init__()
        self.d = d
        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d*d)
        )

    def forward(self, x, t_normalized):
        batch_size = x.shape[0]
        x_with_time = torch.cat([x, t_normalized], dim=1)
        output = self.net(x_with_time)
        G = output.view(batch_size, self.d, self.d)

        # 1. Make symmetric
        G = 0.5 * (G + G.transpose(-1, -2))

        # 2. Make row sums = 0
        row_sums = torch.sum(G, dim=-1, keepdim=True)
        G = G - torch.diag_embed(row_sums.squeeze(-1))

        return G

class ANet(nn.Module):
    def __init__(self, d, hidden_dim=64):
        super(ANet, self).__init__()
        self.d = d

```

```

        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d)
        )

    def forward(self, x, t_normalized):
        x_with_time = torch.cat([x, t_normalized], dim=1)
        output = self.net(x_with_time)

        # make row sum = 0
        mean_output = torch.mean(output, dim=1, keepdim=True)
        constrained_output = output - mean_output

        return constrained_output

class Deep2BSDE(nn.Module):

    def __init__(self, d, T, N, xi, g_func):
        super(Deep2BSDE, self).__init__()
        self.d = d
        self.T = T
        self.N = N
        self.h = T / N
        self.xi = xi
        self.g = g_func

        # Learnable initial values - reduced standard deviation
        self.Y0 = nn.Parameter(torch.randn(1))
        self.Z0 = nn.Parameter(torch.randn(1, d) * 0.1)

        self.g_net = GNet(d)
        self.a_net = ANet(d)

        # Initial strategy parameters - reduced standard deviation
        self.g0 = self._init_constrained_G()
        self.a0 = self._init_constrained_A()

        self.store_trajectory = False # Flag to enable/disable storage
        self.numerical_controls = []
        self.time_points = []

    def _init_constrained_G(self):
        """Initialize G0 to be symmetric with row sums = 0"""
        G = torch.randn(self.d, self.d) * 0.1

```

```

        # Make symmetric
        G = 0.5 * (G + G.T)

        # 2. Make row sums = 0
        row_sums = torch.sum(G, dim=-1, keepdim=True)
        G = G - torch.diag_embed(row_sums.squeeze(-1))

    return nn.Parameter(G)

def _init_constrained_A(self):
    """Initialize A0 to sum to 0 (tangent space)"""
    A = torch.randn(1, self.d) * 0.1
    A = A - torch.mean(A, dim=1, keepdim=True) # Ensure sum = 0
    return nn.Parameter(A)

def forward(self, training_scenarios):

    batch_size = training_scenarios.shape[0]
    device = training_scenarios.device

    # Initialize variables
    Y = self.Y0.expand(batch_size, 1)
    Z = self.Z0.expand(batch_size, -1)
    X = self.xi.expand(batch_size, -1)

    # Pre-compute time values to avoid repeated calculations
    time_steps = torch.arange(self.N, device=device, dtype=torch.
↪float32) * self.h

    u_prev = None

    # Clear previous optimal trajectories if storing
    if self.store_trajectory:
        self.numerical_controls = []
        self.time_points = []

    # Main simulation loop
    for n in range(self.N - 1):
        dW = training_scenarios[:, n, :]

        # Get strategy for this time step
        if n == 0:
            G = self.g0.expand(batch_size, -1, -1)
            A = self.a0.expand(batch_size, -1)
        else:

```

```

        t_normalized = torch.full((batch_size, 1), n / self.N,
↪device=device)

        G = self.g_net(X, t_normalized)
        A = self.a_net(X, t_normalized)

        u_optimal = numerical_optimal_control(
            time_steps[n], X, Z, G,
            num_iterations=10, lr=0.05,
            u_init=u_prev # Warm start from previous timestep
        )

        if self.store_trajectory:
            self.numerical_controls.append(u_optimal.clone().detach())
            self.time_points.append(time_steps[n].item())

        # Update equations
        sigma = diffusion(u_optimal, X)

        #find next X
        X_new = X + drift(u_optimal, X) * self.h + sigma * dW
        X_new = project_to_simplex_efficient(X_new)
        dX = X_new - X

        #f using prev X for Y update
        f_val = f_func_batched(time_steps[n], X, Y, Z, G, u_optimal)

        #trace term using prev X for Y update
        sigma_sigma_T = torch.einsum('bi,bj->bij', sigma, sigma)
        trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
↪unsqueeze(-1)

        Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
↪trace_term) * self.h
        Z = Z + A * self.h + torch.bmm(G, dX.unsqueeze(-1)).squeeze(-1)
        X = X_new

        # Store for next iteration warm start
        u_prev = u_optimal

        # Final time step
        dW = training_scenarios[:, -1, :]
        if self.N > 1:
            t_normalized = torch.ones((batch_size, 1), device=device) # t =
↪T

            G = self.g_net(X, t_normalized)
            A = self.a_net(X, t_normalized)
        else:

```

```

        G = self.g0.expand(batch_size, -1, -1)
        A = self.a0.expand(batch_size, -1)

        # Final optimal control with warm start
        u_optimal = numerical_optimal_control(
            time_steps[-1], X, Z, G,
            num_iterations=5, lr=0.1,
            u_init=u_prev
        )

        # Update equations
        sigma = diffusion(u_optimal, X)

        #find next X
        X_new = X + drift(u_optimal, X) * self.h + sigma * dW
        X_new = project_to_simplex_efficient(X_new)
        dX = X_new - X

        #f using prev X for Y update
        f_val = f_func_batched(time_steps[-1], X, Y, Z, G, u_optimal)

        #trace term using prev X for Y update
        sigma_sigma_T = torch.einsum('bi,bj->bij', sigma, sigma)
        trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
        ↪unsqueeze(-1)

        Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
        ↪trace_term) * self.h
        X = X_new

        return Y, X

    def loss(self, training_scenarios):
        Y_terminal, X_terminal = self.forward(training_scenarios)
        target = self.g(X_terminal)
        return torch.mean((Y_terminal - target)**2)

    def set_research_style():
        plt.rcParams.update({
            'font.family': 'serif',
            'font.size': 11,
            'axes.labelsize': 12,
            'axes.titlesize': 13,
            'xtick.labelsize': 10,
            'ytick.labelsize': 10,
            'legend.fontsize': 10,
            'axes.linewidth': 0.8,

```

```

        'axes.grid': True,
        'grid.alpha': 0.3,
        'lines.linewidth': 1.5,
        'figure.figsize': (8, 6),
        'figure.dpi': 300,
        'savefig.transparent': True,
        'figure.facecolor': 'none'
    })

def train_deep2bsde(model, num_epochs=2000, batch_size=64, lr=0.01):

    # Use Adam with weight decay for better convergence
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-5)

    # Learning rate scheduler
    scheduler = optim.lr_scheduler.ReduceLRonPlateau(optimizer,
    ↪patience=500, factor=0.8)

    best_loss = float('inf')
    loss_history = []

    for epoch in range(num_epochs):
        training_scenarios = brownian_training_data(batch_size, model.N,
    ↪model.d, model.T)

        # Training step
        optimizer.zero_grad()
        loss = model.loss(training_scenarios)
        loss.backward()

        # Gradient clipping for stability
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)

        optimizer.step()
        scheduler.step(loss)

        loss_history.append(loss.item())

        # Track best loss
        if loss.item() < best_loss:
            best_loss = loss.item()

        percentage = (epoch + 1) / num_epochs * 100
        print(f"\rProgress: {percentage:5.1f}%", end='', flush=True)

    print()

```

```

    return model, loss_history

def plot_loss_history(loss_history, run_number):
    """Plot loss history for a single run."""
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 5), facecolor='none')
    ax.patch.set_facecolor('none')

    epochs = np.arange(1, len(loss_history) + 1)
    ax.semilogy(epochs, loss_history, 'b-', linewidth=1.5, alpha=0.8)

    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss (log scale)')
    ax.set_title(f'Training Loss Convergence - Run {run_number}')
    ax.grid(True, alpha=0.3)

    print(f"{np.mean(loss_history[-100:]):.2e}")

    plt.tight_layout()
    plt.show()

def create_results_table(analytical_val, numerical_val, title="Results"):
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Deep2BSDE approximation', f'{numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
        ['Relative error (%)', f'{rel_error:.4f}']
    ]

    table = ax.table(cellText=data,
                    colLabels=['Quantity', 'Value'],
                    cellLoc='left',
                    loc='center',
                    colWidths=[0.6, 0.4])

    table.auto_set_font_size(False)

```

```

table.set_fontsize(11)
table.scale(1, 2)

# Style the table
for i in range(len(data) + 1):
    for j in range(2):
        cell = table[(i, j)]
        if i == 0: # Header
            cell.set_facecolor('#E6E6FA')
            cell.set_text_props(weight='bold')
        else:
            cell.set_facecolor('#F8F8FF')
            cell.set_edgecolor('black')
            cell.set_linewidth(0.5)

plt.title(title, fontsize=13, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

def create_summary_table(analytical_val, mean_numerical_val, title="Summary_
→Results"):
    """Create summary table with mean results."""
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(mean_numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Mean Deep2BSDE approximation', f'{mean_numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
        ['Relative error (%)', f'{rel_error:.4f}']
    ]

    table = ax.table(cellText=data,
                     colLabels=['Quantity', 'Value'],
                     cellLoc='left',
                     loc='center',
                     colWidths=[0.6, 0.4])

    table.auto_set_font_size(False)
    table.set_fontsize(11)

```

```

table.scale(1, 2)

# Style the table
for i in range(len(data) + 1):
    for j in range(2):
        cell = table[(i, j)]
        if i == 0: # Header
            cell.set_facecolor('#E6E6FA')
            cell.set_text_props(weight='bold')
        else:
            cell.set_facecolor('#F8F8FF')
            cell.set_edgecolor('black')
            cell.set_linewidth(0.5)

plt.title(title, fontsize=13, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

```

```

def plot_control_vs_time(model, num_paths=3):
    set_research_style()

    model.store_trajectory = True
    scenarios = brownian_training_data(num_paths, model.N, model.d, model.T)
    model.forward(scenarios)
    model.store_trajectory = False

    fig, ax = plt.subplots(1, 1, figsize=(10, 4), facecolor='none')
    ax.patch.set_facecolor('none') # Add this line back

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c']

    for path in range(num_paths):
        color = colors[path % len(colors)]
        numerical_u = [controls[path, 0].item() for controls in model.
↪ numerical_controls]

        ax.plot(model.time_points, numerical_u, '-', color=color,
                label=f'Deep2BSDE (Path {path + 1})', linewidth=1.5, alpha=0.
↪ 8)

    ax.set_xlabel('Time $t$')
    ax.set_ylabel('Optimal Control $u^*(t)$')
    ax.set_title('Numerical Optimal Control')
    ax.legend(frameon=True, fancybox=False, shadow=False, framealpha=0.9)
    ax.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

```
def test_multiple_runs(num_runs=5, d=1, T=1, N=20, num_epochs=2000):
    """Run multiple experiments with individual results and summary."""
    xi = torch.tensor([1.0]).repeat(d)/d

    def analytical_solution_value():
        return (1/Beta)*torch.sum(xi * support)**2

    def g_func(x):
        return (1/Beta)*(x @ support)**2

    final_values = []
    analytical_val = analytical_solution_value()

    print(f"Starting Deep2BSDE experiments: {num_runs} runs, {num_epochs}
↪epochs each")
    print("=" * 60)

    # Run each experiment
    for run in range(num_runs):

        model = Deep2BSDE(d, T, N, xi, g_func)

        trained_model, loss_history = train_deep2bsde(model,
↪num_epochs=num_epochs, batch_size=64, lr=0.005)

        final_Y0 = trained_model.Y0.item()
        final_values.append(final_Y0)

        # Plot loss for this run
        plot_loss_history(loss_history, run + 1)

        # Show results table for this run
        create_results_table(analytical_val, final_Y0, f"Run {run + 1}
↪Results")

        plot_control_vs_time(trained_model,1)

        # Summary table with mean results
        mean_numerical_val = np.mean(final_values)
        create_summary_table(analytical_val, mean_numerical_val, "Summary of All
↪Runs")

    if __name__ == "__main__":
```

```
test_multiple_runs(num_runs=1, d=3, num_epochs=500)
```

A.6 Merton Example

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
```

```
# SPY-based parameters for Merton problem
mu = 0.1200      # 12% expected annual return
r = 0.0400      # 4% risk-free rate
sigma = 0.1800   # 18% annual volatility
gamma = 0.5      # Utility parameter
```

```
def brownian_training_data(batch_size, N, d, T):

    dt = T / N
    sqrt_dt = np.sqrt(dt)

    # Generate all random numbers at once with correct scale
    training_scenarios = np.random.normal(0, sqrt_dt, (batch_size, N, 1))

    return torch.tensor(training_scenarios, dtype=torch.float32).
    ↪expand(batch_size, N, d)
```

```
def drift(u, x):
    return r * x + u * (mu - r)

def diffusion(u, x):
    batch_size, d = x.shape
    # For Merton:  $\sigma(\pi, X) = \pi\sigma$ , so diffusion matrix is  $\pi\sigma * I$ 
    sigma_matrix = torch.eye(d, device=x.device).unsqueeze(0).
    ↪repeat(batch_size, 1, 1)
    sigma_vals = u * sigma #  $\pi\sigma$ 
    return sigma_vals.unsqueeze(-1) * sigma_matrix
```

```
Beta = 0
```

```
def cost_tensor(x, u):
    return 0 * torch.sum(u * u, dim=-1, keepdim=True)
```

```
def analytical_optimal_control(t, X, T): # Need to pass T
    time_factor = np.exp(-r * (T - t))
```

```

    optimal_control = (mu - r) / (gamma * sigma**2) * time_factor
    batch_size = X.shape[0]
    return torch.full((batch_size, 1), optimal_control, device=X.device,
↳dtype=X.dtype)

def analytical_solution_value(T, x0=1.0):
    first_term = -((mu - r)**2) * T / (2 * sigma**2)
    second_term = -gamma * x0 * np.exp(r * T)
    return np.exp(first_term + second_term)

```

```

def numerical_optimal_control(t, X, Z, Gamma, num_iterations=20, lr=0.01,
↳u_init=None):

    X_detached = X.detach()
    Z_detached = Z.detach()
    Gamma_detached = Gamma.detach()

    batch_size, d = X_detached.shape

    # Warm start from previous solution if provided
    if u_init is not None:
        u = u_init.detach().clone().requires_grad_(True)
    else:
        u = torch.zeros(batch_size, 1, requires_grad=True)

    # Use Adam optimizer for better convergence
    optimizer = torch.optim.Adam([u], lr=lr, eps=1e-8)

    for _ in range(num_iterations):
        optimizer.zero_grad()

        # Compute objective function
        cost_term = cost_tensor(X_detached, u) # (batch_size, 1)

        # Drift term:  $b(u, x) \cdot Z$ 
        drift_vals = drift(u, X_detached) # (batch_size, d)
        drift_term = torch.sum(drift_vals * Z_detached, dim=-1,
↳keepdim=True) # (batch_size, 1)

        # Diffusion term:  $(1/2) \cdot \sigma^T(u, x) \cdot \text{trace}(\text{Gamma})$ 
        sigma = diffusion(u, X_detached) # (batch_size, d)
        sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
        trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T,
↳Gamma_detached).unsqueeze(-1)
        diffusion_term = 0.5 * trace_vals

        # Total objective to minimize

```

```

        objective = cost_term + drift_term + diffusion_term
        loss = torch.mean(objective) # Average over batch
        loss.backward()
        optimizer.step()

    return u.detach()

# Fully batched f_func
def f_func_batched(t, X, Y, Z, Gamma, u_optimal):

    # Compute objective function
    cost_term = cost_tensor(X, u_optimal) # (batch_size, 1)

    # Drift term:  $b(u, x) \cdot Z$ 
    drift_vals = drift(u_optimal, X) # (batch_size, d)
    drift_term = torch.sum(drift_vals * Z, dim=-1, keepdim=True) #  $\rightarrow$  (batch_size, 1)

    # Diffusion term:  $(1/2) \sigma \sigma^T(u, x) \text{trace}(\Gamma)$ 
    sigma = diffusion(u_optimal, X) # (batch_size, d)
    sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
    trace_vals = torch.einsum('bij,bji->b', sigma_sigma_T, Gamma).
 $\rightarrow$  unsqueeze(-1)
    diffusion_term = 0.5 * trace_vals

    # Total objective to minimize
    objective = cost_term + drift_term + diffusion_term

    return Beta*Y - objective

# Time-aware neural networks
class GNet(nn.Module):
    def __init__(self, d, hidden_dim=64):
        super(GNet, self).__init__()
        self.d = d
        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d*d)
        )

    def forward(self, x, t_normalized):
        batch_size = x.shape[0]
        x_with_time = torch.cat([x, t_normalized], dim=1)
        output = self.net(x_with_time)

```

```

G = output.view(batch_size, self.d, self.d)

if self.d == 1:
    # 1D case: just ensure G is positive
    G = torch.exp(G)
else:
    # Multi-dimensional case: ensure positive definite
    # Make symmetric first
    G = 0.5 * (G + G.transpose(-1, -2))

    # Eigenvalue decomposition
    eigenvals, eigenvecs = torch.linalg.eigh(G)

    # Force all eigenvalues to be positive
    eigenvals = torch.exp(eigenvals) # Ensures all positive

    # Reconstruct:  $G = Q * \text{diag}(\lambda) * Q^T$ 
    G = torch.bmm(
        torch.bmm(eigenvecs, torch.diag_embed(eigenvals)),
        eigenvecs.transpose(-1, -2)
    )

return G

class ANet(nn.Module):
    def __init__(self, d, hidden_dim=64):
        super(ANet, self).__init__()
        self.d = d
        self.net = nn.Sequential(
            nn.Linear(d + 1, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, d)
        )

    def forward(self, x, t_normalized):
        x_with_time = torch.cat([x, t_normalized], dim=1)
        output = self.net(x_with_time)

        return output

class Deep2BSDE(nn.Module):
    def __init__(self, d, T, N, xi, g_func):
        super(Deep2BSDE, self).__init__()
        self.d = d

```

```

self.T = T
self.N = N
self.h = T / N
self.xi = xi
self.g = g_func

# Learnable initial values - reduced standard deviation
self.Y0 = nn.Parameter(torch.randn(1))
self.Z0 = nn.Parameter(torch.randn(1, d) * 0.1)

self.g_net = GNet(d)
self.a_net = ANet(d)

# Initial strategy parameters - reduced standard deviation
if d == 1:
    # 1D case: just a positive scalar
    self.g0 = nn.Parameter(torch.tensor([[0.1]]))
else:
    # Multi-dimensional case: negative definite matrix
    G_init = torch.randn(d, d) * 0.1
    G_init = 0.5 * (G_init + G_init.T) # Make symmetric
    eigenvals, eigenvecs = torch.linalg.eigh(G_init)

    eigenvals = torch.exp(eigenvals * 0.1) # Small POSITIVE
→ eigenvalues

    # Reconstruct:  $G = Q * \text{diag}(\lambda) * Q^T$ 
    G_init = eigenvecs @ torch.diag(eigenvals) @ eigenvecs.T
    self.g0 = nn.Parameter(G_init)

self.a0 = nn.Parameter(torch.randn(1, d) * 0.1)

self.store_trajectory = False # Flag to enable/disable storage
self.numerical_controls = []
self.analytical_controls = []
self.time_points = []

def forward(self, training_scenarios):

    batch_size = training_scenarios.shape[0]
    device = training_scenarios.device

    # Initialize variables
    Y = self.Y0.expand(batch_size, 1)
    Z = self.Z0.expand(batch_size, -1)
    X = self.xi.expand(batch_size, -1)

```

```

        # Pre-compute time values to avoid repeated calculations
        time_steps = torch.arange(self.N, device=device, dtype=torch.
→float32) * self.h

        u_prev = None

        # Clear previous optimal trajectories if storing
        if self.store_trajectory:
            self.numerical_controls = []
            self.analytical_controls = []
            self.time_points = []

        # Main simulation loop
        for n in range(self.N - 1):
            dW = training_scenarios[:, n, :]

            # Get strategy for this time step
            if n == 0:
                G = self.g0.expand(batch_size, -1, -1)
                A = self.a0.expand(batch_size, -1)
            else:
                t_normalized = torch.full((batch_size, 1), n / self.N,
→device=device)
                G = self.g_net(X, t_normalized)
                A = self.a_net(X, t_normalized)

            if n==0:
                u_optimal = numerical_optimal_control(
                    time_steps[n], X, Z, G,
                    num_iterations=100, lr=0.05,
                    u_init=u_prev # Warm start from previous timestep
                )
            else:
                u_optimal = numerical_optimal_control(
                    time_steps[n], X, Z, G,
                    num_iterations=10, lr=0.05,
                    u_init=u_prev # Warm start from previous timestep
                )

            if self.store_trajectory:
                self.numerical_controls.append(u_optimal.clone().detach())
                analytical_control =
→analytical_optimal_control(time_steps[n].item(), X, self.T)
                self.analytical_controls.append(analytical_control.clone().
→detach())

                self.time_points.append(time_steps[n].item())

```

```

        # Update equations
        sigma = diffusion(u_optimal, X)

        #find next X
        X_new = X + drift(u_optimal, X) * self.h + torch.bmm(sigma, dW.
→unsqueeze(-1)).squeeze(-1)
        dX = X_new - X

        #f using prev X for Y update
        f_val = f_func_batched(time_steps[n], X, Y, Z, G, u_optimal)

        #trace term using prev X for Y update
        sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
        trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
→unsqueeze(-1)

        Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
→trace_term) * self.h
        Z = Z + A * self.h + torch.bmm(G, dX.unsqueeze(-1)).squeeze(-1)
        X = X_new

        # Store for next iteration warm start
        u_prev = u_optimal

    # Final time step
    dW = training_scenarios[:, -1, :]
    if self.N > 1:
        t_normalized = torch.ones((batch_size, 1), device=device) # t =
→T

        G = self.g_net(X, t_normalized)
        A = self.a_net(X, t_normalized)
    else:
        G = self.g0.expand(batch_size, -1, -1)
        A = self.a0.expand(batch_size, -1)

    # Final optimal control with warm start
    u_optimal = numerical_optimal_control(
        time_steps[-1], X, Z, G,
        num_iterations=5, lr=0.1,
        u_init=u_prev
    )

    if self.store_trajectory:
        self.numerical_controls.append(u_optimal.clone().detach())
        analytical_control = analytical_optimal_control(time_steps[-1].
→item(), X, self.T)

```

```

        self.analytical_controls.append(analytical_control.clone()).
        detach()

        self.time_points.append(time_steps[n].item())

        # Update equations
        sigma = diffusion(u_optimal, X)

        #find next X
        X_new = X + drift(u_optimal, X) * self.h + torch.bmm(sigma, dW.
        unsqueeze(-1)).squeeze(-1)
        dX = X_new - X

        #f using prev X for Y update
        f_val = f_func_batched(time_steps[-1], X, Y, Z, G, u_optimal)

        #trace term using prev X for Y update
        sigma_sigma_T = torch.bmm(sigma, sigma.transpose(1, 2))
        trace_term = 0.5 * torch.einsum('bij,bji->b', sigma_sigma_T, G).
        unsqueeze(-1)

        Y = Y + torch.sum(Z * dX, dim=1, keepdim=True) + (f_val +
        trace_term) * self.h
        X = X_new

    return Y, X

def loss(self, training_scenarios):
    Y_terminal, X_terminal = self.forward(training_scenarios)
    target = self.g(X_terminal)
    return torch.mean((Y_terminal - target)**2)

def set_research_style():
    plt.rcParams.update({
        'font.family': 'serif',
        'font.size': 11,
        'axes.labelsize': 12,
        'axes.titlesize': 13,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
        'legend.fontsize': 10,
        'axes.linewidth': 0.8,
        'axes.grid': True,
        'grid.alpha': 0.3,
        'lines.linewidth': 1.5,
        'figure.figsize': (8, 6),
        'figure.dpi': 300,
        'savefig.transparent': True,
    })

```

```

        'figure.facecolor': 'none'
    })

def train_deep2bsde(model, num_epochs=2000, batch_size=64, lr=0.01):

    # Use Adam with weight decay for better convergence
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-5)

    # Learning rate scheduler
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
↪patience=500, factor=0.8)

    best_loss = float('inf')
    loss_history = []

    for epoch in range(num_epochs):
        training_scenarios = brownian_training_data(batch_size, model.N,
↪model.d, model.T)

        # Training step
        optimizer.zero_grad()
        loss = model.loss(training_scenarios)
        loss.backward()

        # Gradient clipping for stability
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)

        optimizer.step()
        scheduler.step(loss)

        loss_history.append(loss.item())

        # Track best loss
        if loss.item() < best_loss:
            best_loss = loss.item()

        percentage = (epoch + 1) / num_epochs * 100
        print(f"\rProgress: {percentage:5.1f}%", end='', flush=True)

    print()

    return model, loss_history

def plot_loss_history(loss_history, run_number):
    """Plot loss history for a single run."""
    set_research_style()

```

```

fig, ax = plt.subplots(figsize=(8, 5), facecolor='none')
ax.patch.set_facecolor('none')

epochs = np.arange(1, len(loss_history) + 1)
ax.semilogy(epochs, loss_history, 'b-', linewidth=1.5, alpha=0.8)

ax.set_xlabel('Epoch')
ax.set_ylabel('Loss (log scale)')
ax.set_title(f'Training Loss Convergence - Run {run_number}')
ax.grid(True, alpha=0.3)

print(f"{np.mean(loss_history[-100:]):.2e}")

plt.tight_layout()
plt.show()

def create_results_table(analytical_val, numerical_val, title="Results"):
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Deep2BSDE approximation', f'{numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
        ['Relative error (%)', f'{rel_error:.4f}']
    ]

    table = ax.table(cellText=data,
                     colLabels=['Quantity', 'Value'],
                     cellLoc='left',
                     loc='center',
                     colWidths=[0.6, 0.4])

    table.auto_set_font_size(False)
    table.set_fontsize(11)
    table.scale(1, 2)

    # Style the table
    for i in range(len(data) + 1):
        for j in range(2):

```

```

        cell = table[(i, j)]
        if i == 0: # Header
            cell.set_facecolor('#E6E6FA')
            cell.set_text_props(weight='bold')
        else:
            cell.set_facecolor('#F8F8FF')
            cell.set_edgecolor('black')
            cell.set_linewidth(0.5)

plt.title(title, fontsize=13, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

def create_summary_table(analytical_val, mean_numerical_val, title="Summary_
↳Results"):
    """Create summary table with mean results."""
    set_research_style()

    fig, ax = plt.subplots(figsize=(8, 3), facecolor='none')
    ax.patch.set_facecolor('none')
    ax.axis('tight')
    ax.axis('off')

    error = abs(mean_numerical_val - analytical_val)
    rel_error = error / abs(analytical_val) * 100

    data = [
        ['Analytical solution', f'{analytical_val:.8f}'],
        ['Mean Deep2BSDE approximation', f'{mean_numerical_val:.8f}'],
        ['Absolute error', f'{error:.2e}'],
        ['Relative error (%)', f'{rel_error:.4f}']
    ]

    table = ax.table(cellText=data,
                     colLabels=['Quantity', 'Value'],
                     cellLoc='left',
                     loc='center',
                     colWidths=[0.6, 0.4])

    table.auto_set_font_size(False)
    table.set_fontsize(11)
    table.scale(1, 2)

    # Style the table
    for i in range(len(data) + 1):
        for j in range(2):
            cell = table[(i, j)]

```

```

        if i == 0: # Header
            cell.set_facecolor('#E6E6FA')
            cell.set_text_props(weight='bold')
        else:
            cell.set_facecolor('#F8F8FF')
            cell.set_edgecolor('black')
            cell.set_linewidth(0.5)

plt.title(title, fontsize=13, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

```

```

def plot_control_vs_time(model, num_paths=3):
    set_research_style()

    model.store_trajectory = True
    scenarios = brownian_training_data(num_paths, model.N, model.d, model.T)
    model.forward(scenarios)
    model.store_trajectory = False

    fig, ax = plt.subplots(1, 1, figsize=(10, 4), facecolor='none')
    ax.patch.set_facecolor('none') # Add this line back

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c']

    for path in range(num_paths):
        color = colors[path % len(colors)]
        numerical_u = [controls[path, 0].item() for controls in model.
↪numerical_controls]
        analytical_u = [controls[path, 0].item() for controls in model.
↪analytical_controls]

        ax.plot(model.time_points, numerical_u, '-', color=color,
                label=f'Deep2BSDE (Path {path + 1})', linewidth=1.5, alpha=0.
↪8)
        ax.plot(model.time_points, analytical_u, '--', color=color,
                label=f'Analytical (Path {path + 1})', linewidth=1.2,
↪alpha=0.9)

        ax.set_xlabel('Time $t$')
        ax.set_ylabel('Optimal Control $u^*(t)$')
        ax.set_title('Numerical Optimal Control')
        ax.legend(frameon=True, fancybox=False, shadow=False, framealpha=0.9)
        ax.grid(True, alpha=0.3)

    plt.tight_layout()

```

```
plt.show()
```

```
def test_multiple_runs(num_runs=5, d=1, T=1, N=20, num_epochs=2000):
    """Run multiple experiments with individual results and summary."""
    xi = torch.ones(1, d) * 1.0 # Initial wealth = $1

    def g_func(x):
        # Terminal utility:  $U(X_T) = -\exp(-\gamma X_T)$ 
        return torch.exp(-gamma * x)

    final_values = []
    analytical_val = analytical_solution_value(T, xi.mean().item())

    print(f"Starting Deep2BSDE experiments: {num_runs} runs, {num_epochs} epochs each")
    print("=" * 60)

    # Run each experiment
    for run in range(num_runs):

        model = Deep2BSDE(d, T, N, xi, g_func)

        trained_model, loss_history = train_deep2bsde(model, num_epochs=num_epochs, batch_size=64, lr=0.005)

        final_Y0 = trained_model.Y0.item()
        final_values.append(final_Y0)

        # Plot loss for this run
        plot_loss_history(loss_history, run + 1)

        # Show results table for this run
        create_results_table(analytical_val, final_Y0, f"Run {run + 1}")

        plot_control_vs_time(trained_model, 1)

    # Summary table with mean results
    mean_numerical_val = np.mean(final_values)
    create_summary_table(analytical_val, mean_numerical_val, "Summary of All Runs")

    if __name__ == "__main__":
        test_multiple_runs(num_runs=1, d=1, num_epochs=500)
```

References

- [1] C. Beck, W. E, and A. Jentzen. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *Journal of Nonlinear Science*, 29(4):1563–1619, 2019.
- [2] P. Cheridito, H. M. Soner, N. Touzi, and N. Victoir. Second-order backward stochastic differential equations and fully nonlinear parabolic pdes. *Communications on Pure and Applied Mathematics*, 60(7):1081–1110, 2007.
- [3] S. E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer Finance. Springer, New York, 2004.
- [4] A. M. G. Cox, S. Källblad, C. Wang *A Measure-valued HJB Perspective On Bayesian Optimal Adaptive Control*.
- [5] A. M. G. Cox, S. Källblad, M. Larsson, S. Svaluto-Ferro *Controlled measure-valued martingales: a viscosity solution approach*.
- [6] Á. Cartea, S. Jaimungal, and J. Peñalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, Cambridge, 2015.